# ComCE: Efficient Community-Aware Counterfactual Explanations for Graph Neural Networks on Large Graphs

Anonymous Author(s)

## ABSTRACT

Graph neural networks (GNNs) are powerful graph-based deep-learning models that have gained significant attention and demonstrated remarkable performance in various domains, including drug discovery and recommendation systems. Owing to their popularity, there is an increasing need to explain GNN predictions since GNNs are *black-box* machine learning models. One way to address this is *counterfactual explanation* whose objective is to *change the GNN prediction by minimal changes in the input graph.* However, existing counterfactual explanation methods are *ineffective* and *inefficient* due to the following two major limitations: (1) most of them depend on the masking techniques on the input graph, which are not easy-to-interpret; and (2) all of them requires expensive time cost since they need to find explanations via heavy search processing under an optimization problem. In this paper, we propose ComCE, a novel approach by utilizing communities under the input graph, reducing the searching process and further improving the efficiency. Extensive experiments on real-life graphs demonstrated that the counterfactual explanation from ComCE provides necessary changes for model behavior and outperforms state-of-the-art methods across major metrics.

## 1 INTRODUCTION

*Counterfactual explanations* for Graph Neural Networks (GNNs) have become increasingly important as GNNs are widely adopted in high-stakes domains such as biomedical analysis [23, 32, 35, 46], financial risk control [26, 33], and recommender systems [10, 45]. Unlike post-hoc *factual explanations* that only highlight influential nodes or edges, counterfactual explanations aim to answer *what minimal change to the graph would alter the model's prediction*, offering actionable and causal insights into model reasoning. Recent studies have explored different strategies to achieve this goal. Specifically, heuristic-based methods (*e.g.,*, DDBS/OBS [2], MACCS [43]) use rule-guided graph perturbations specific to domains such as molecules or brain networks; search-based methods (*e.g.,* GNN-MOExp [22]) identify counterfactual subgraphs by comparing structural similarity within the dataset; and learning-based methods (*e.g.,* CF-GNNExplainer [24], RCExplainer [3], GNNAdv [36], CLEAR [27]) employ optimization, reinforcement learning, or generative models to modify graph structures or features.

Despite progress, these approaches largely operate at individual nodes or edges, which imposes several critical limitations. (1) **Lack of group awareness**. By focusing on single-node or single-edge perturbations, these approaches *overlook* higher-order dependencies that emerge from group interactions. For instance, in a social network, removing one or two friendship links offers limited insight into the collective behavior of an entire *interest circle* that drives the classification outcome (*e.g.,* movie genre preference or political alignment). (2) **Dense but fragmented explanations**. Explanations derived from such low-level perturbations are often *dense and hard to interpret, e.g.,* a single counterfactual explanation may involve editing dozens or even hundreds of edges, resulting in explanations too fragmented to convey meaningful patterns. (3) **Inefficient computations**. These node-level operations incur high computational costs; searching for valid counterfactuals requires iteratively evaluating numerous local modifications, which becomes prohibitive for large-scale graphs. (4) **Fragile robustness**. As they depend on local connectivity, these methods suffer from robustness issues—minor noise or irrelevant connections can easily distort the derived explanation, undermining reliability in real-world networks.

Motivated by these limitations, we propose a novel *community-aware* method for counterfactual explanations, ComCE, which decouples the problem into *where to search* and *what to delete.* In its offline phase, ComCE transforms the receptive field of a target vertex into an overlapping explanatory community space in graphs and learns to select the communities most relevant to the target, thereby producing a compact search domain. In its online phase, ComCE searches only within the selected communities to locate the minimal edge set that flips the prediction, using a *lightweight world model* for cheap scoring and reserving the real GNN for a bounded number of certification queries. This design offers the following features.

(1) **Group-centric explanations.** By identifying and reasoning over community structures, our method directly explains how clusters of related nodes, *e.g.,* user socially groups or citation communities, collectively influence the predictions of GNNs.
(2) **Simplified interpretability.** Communities serve as intermediate abstractions, simplifying complex graph structures into concise community-level rules (*e.g.,* Sci-fi community users prefer action movies), thereby reducing explanation complexity.
(3) **High efficiency.** Rather than searching over the entire receptive field, ComCE first narrows the search to a few relevant communities and then refines to the minimal edge set inside them, scoring candidates with a cheap world model and invoking the expensive GNN only within a fixed query budget.
(4) **Improved robustness.** Because community structures encapsulate global connectivity patterns, they remain stable under noisy perturbations, ensuring consistent and trustworthy explanations across graph types and domains.

*Contributions.* The following shows our contributions.

(1) *Community-aware counterfactual formulation.* We formulate counterfactual explanation as a group-level perturbation problem and design a community-aware cost function that unifies edge-level and community-level modifications. We further prove that finding the minimum-cost explanation is NP-hard.
(2) *Heuristic baseline with complexity guarantee.* We present a simple greedy heuristic that ranks candidate perturbations by their normalized PPR-based importance to approximate the MCE solution, and provide a theoretic time complexity analysis showing that it remains tractable for million-scale graphs.

(3) *A two-stage learning framework.* To remedy the weaknesses of the heuristic approach, we propose a two-stage framework ComCE. In the *offline phase*, we cast the receptive field as an overlapping explanatory community space and learn a *Set Transformer selector* SetT to produce a compact community-induced search domain. In the *online phase*, a *Dreamer-style planner* searches this domain, where a lightweight world model scores candidate perturbations and the real GNN is reserved for a bounded number of certification queries.
(4) *Comprehensive experiments.* Extensive experiments on real-life graphs demonstrate the superiority of ComCE over state-of-the-arts in terms of effectiveness, sparsity, efficiency, and robustness.

## 2 RELATED WORK

Prior methods for explaining GNN-based models are classified into two types: (1) factual explanation methods, which aim to extract meta-paths/subgraphs and/or features that are with the maximum influence over the prediction; and (2) counterfactual explanation methods, which aim to find the smallest change in the input graph that changes the model's prediction.

*Factual explanation methods.* These methods provide the explanations by utilizing *either* (1) the parameters inside GNN models, *or* (2) the surrogate models that are simple and interpretable, *or* (3) the local information extracted from origin graph. Since the internal parameters of GNN models are not always accessible in real world, such methods, *e.g.,* CAM [31] and Sensitivity Analysis (SA) [4], are not suitable for explaining black-box GNNs. Among the surrogate-based methods, GraphLime [16] is used most frequently which selects vertices and its $K$-hop neighborhood by utilizing Hilbert-Schmidt Independence Criterion Lasso [47]. However, it assumes that the vertex features in the origin graph are easily interpretable, which is not true in reality. The methods of the third type find *important subgraphs* as explanations by firstly perturbing the input graph data and then iteratively selecting the one with the maximum information gain, *e.g.,* GNNExplainer [48], GraphMask [34], PGExplainer [25] and SubgraphX [50]. Among these, GNNExplainer extracts important subgraphs and vertex features as factual explanations by maximizing the mutual information between subgraphs and predictions; PGExplainer provides subgraphs as factual explanations by adopting a deep neural network to parameterize the generation of explanations; and SubgraphX returns a subgraph as factual explanations based on Shapley values. Zorro [9] uses a greedy selection strategy to find explanations in the form of important nodes and features that maximizes Fidelity.

*Counterfactual explanation methods.* These methods can be divided into three primary classes. (1) *Heuristic-based methods* that rely on a rigorous policy of modifying the input graph until they reach a valid counterfactual explanation, *e.g.,* DDBS/OBS [2] tailored for brain networks by using bidirectional edge perturbations and MACCS [43] by leveraging molecular domain rules via SELFIES perturbations. However, both methods are *domain-specific* and cannot be transferred to other domains like social graphs. (2) *Search-based methods* that rely on a specific criterion (*e.g.,* similarity between instances) to search for a counterfactual explanation within the dataset for a given input instance, *e.g.,* GNN-MOExp [22] searching for a subgraph in the original instance that optimizes the counterfactual features based on its corresponding factual explanation. (3) *Learning-based methods* that study the output variations w.r.t. input changes, further split into the following three types. (i) *Perturbation methods* that learns a soft-mask that represents which sub-structures and/or node features should be present and which removed to get a counterfactual explanation, *e.g.,* CF-GNNExplainer [24], masking edges via sparse optimization, RCExplainer [3], using decision boundaries to remove critical subgraphs, $CF^2$ [37] and NSEG [5] by eliminating the factual subgraph of a specific input instance from original graph. (ii) *Reinforcement learning methods* that utilize traditional reinforcement learning algorithm to produce explanations that maximize a user-defined cumulative reward function, *e.g.,* MEG [30] for molecules and MACDA [29] for drug-target pairs, both of which are *domain-specific* as well. (iii) *Generative methods* that samples the counterfactual explanation from a learned latent space, *e.g.,* GNNAdv [36] exploiting adversarial attacks on a *topology attribution map* and CLEAR [27] employing *variational autoencoders* (VAEs); However, the learned latent space is opaque and may produce *unrealistic modifications*.

Unlike the previous methods focus on providing the local explanations for a specific data point, GCFExplainer [17] gives the global counterfactual explanations for entire datasets by adopting heuristic meta-graph traversals.

## 3 PRELIMINARIES

This section first introduces the background of general GNN models, and then formally defines the counterfactual explanation problem.

**Basic framework of GNN models.** Given a graph $G(V, E)$ where $V$ and $E$ are the sets of vertices and edges, respectively, we denote the initial representation of a vertex $u$ as $\mathbf{e}_u^{(0)}$ whose value is usually either assigned as its attribute vector $\mathbf{f}_u$ or randomly initialized. To get the embedding of vertex $u$ at layer $l$, a $L$-layer GNN model $\mathcal{M}$ performs the computations as follows:

$$\mathbf{e}_u^{(l)} = g(\mathbf{e}_u^{(l-1)}, \bigoplus\nolimits_{v \in \mathcal{N}_u} f(\mathbf{e}_u^{(l-1)}, \mathbf{e}_v^{(l-1)})), \tag{1}$$

where $f(\cdot)$, $\bigoplus(\cdot)$, and $g(\cdot)$ are customized functions (*i.e.,* neural network modules) for *calculating* messages from each neighbour $v \in \mathcal{N}_u$ of vertex $u$, *aggregating* the messages of all neighbors of vertex $u$, and *updating* the representation of vertex $u$, respectively.

For the vertex classification tasks, we denote the number of classes by $T$. Given a graph $G$ and a vertex $u$, the GNN model $\mathcal{M}$ produces a $(T_c \times 1)$ probability vector $\mathbf{y}_u$ where each element is the probability that vertex $u$ belongs to each of $T_c$ classes. Usually, vertex $u$ is predicted to be class $t$ if $y_u(t)$ is the highest value in $y_u$. When the context is clear, we denote by $t_u = \arg\max_t y_u(t)$ the class that model $\mathcal{M}$ predicts for vertex $u$.

*Remarks.* In this paper, we focus on the vertex classification tasks, but our approaches could be easily extended to graph classification and link prediction tasks. See more experimental results in Section 6.

**Factual Explanation for GNNs.** Given a graph $G$, a GNN model $\mathcal{M}$ and a vertex $u$, we consider a *factual explanation* $\text{exp} = (G_s, \mathbf{f}_s)$ from $G$, where $G_s$ is a sub-graph deduced from $G$ and $\mathbf{f}_s$ is a sub-feature vector deduced from $\mathbf{f}$, such that the model $\mathcal{M}$ can reproduce the classification $t_u$ by using exp only, *i.e.,* $\mathcal{M}(\text{exp}, u) = t_u$. Technically, $G_s$ (resp. $\mathbf{f}_s$) reveals the important topology (resp. features)




**Figure 1: The workflow of** Greedy

that makes model $\mathcal{M}$ produce the same classification $t_u$.

**Counterfactual explanation.** Given a graph $G(V, E)$, a GNN-based classifier $\mathcal{M}$, and a target vertex $u$ with predicted label $t_u = \mathcal{M}(G, u)$, a *counterfactual explanation* is a minimal perturbation $\Delta$ on $G$ such that the prediction on $u$ changes, *i.e.,* $\mathcal{M}(G \setminus \Delta, u) \neq t_u$.

In this work, we focus on *edge deletions* as the primary perturbation, *i.e.,* $\Delta \subseteq E$ removes a set of existing edges in the graph, yielding the perturbed graph $G - \Delta$. Edge deletions are a natural choice for explaining GNN predictions, since the message-passing mechanism propagates information strictly along edges; removing an edge therefore directly reflects a modification of the information flow that drives the predictions of a GNN.

Formally, this paper studies the *minimal counterfactual explanation* (MCE) problem that aims to find

$$\Delta^* = \arg\min_{\Delta} \ \text{cost}(\Delta) \quad \text{s.t.} \quad \mathcal{M}(G \setminus \Delta, u) \neq t_u,$$

where $\text{cost}(\cdot)$ measures the structural effort of perturbation $\Delta$. We define a community-aware cost function in Section 4.

*Why edge deletion rather than addition.* We restrict structural perturbations to edge *deletions* and exclude edge *additions* for two reasons. First, adding edges introduces relationships that do not exist in the observed graph, which easily yields *implausible* explanations (*e.g.,* fabricating a citation between two unrelated papers). This violates the *plausibility* and *actionability* principles [39, 41, 42], which require a counterfactual to be realizable and faithful to the data manifold rather than an arbitrary artifact. Second, the candidate space of additions scales with all non-adjacent vertex pairs, *i.e.,* $O(|V|^2)$, whereas deletions are bounded by $O(|E|)$; admitting additions therefore drastically enlarges the search space and increases the difficulty of finding a minimal counterfactual.

Next, we prove that solving the MCE problem is NP-hard.

**Theorem 1:** *Solving the* MCE *problem is* NP*-hard.* □

**Proof:** We firstly prove a decision version of MCE is NP-complete, and then prove that solving MCE problem is NP-hard.

Firstly, we define a decision version of MCE problem as below: given a graph $G(V, E)$, a GNN-based classifier $\mathcal{M}$, a target vertex $u$, and an integer $k$, decide whether there exists a perturbation set $\Delta \subseteq E$ with $|\Delta| \leq k$ such that $\mathcal{M}(G \setminus \Delta, u) \neq t_u$.

We construct a polynomial-time reduction from the *Hitting Set* problem. Let each element in the universe represent a candidate edge, and let each subset correspond to a structural condition required for preserving $\mathcal{M}(G, u) = t_u$. Flipping the prediction demands removing at least one edge from each subset, equivalent to finding a hitting set of size $\leq k$. Then, we prove that verifying a given $\Delta$ of perturbations is polynomial with forward pass of GNN plus the size check. Hence, the decision version is NP-complete, and solving this optimization problem is NP-hard. □

## 4 A COMMUNITY-AWARE STRATEGY

In this section, we present a community-aware cost function, followed by a greedy strategy to solving MCE problem.

**Challenges**. Since the MCE problem is NP-hard it is prohibitive to enumerate all edge subsets. Instead of perturbing the prediction at the edge level, we focus on explaining at the *community level* since most of real-life graphs have the community structures [18]. By detecting and leveraging community structures, we can efficiently captures global relational patterns and delivers more semantically coherent explanations. However, existing edge-level cost functions cannot measure the cost of a community-level perturbation, and thus it is non-trivial to adopt this community-aware view.

*Input:* $G(V, E)$, vertex $u$, GNN model $\mathcal{M}$, target class $t_u$, restart probability $\alpha$, subgraph size $k$, hyper-parameters $\lambda_1$ and $\lambda_2$.
*Output:* a perturbation set $\Delta$ that flips $\mathcal{M}(G, u)$.

$\Delta := \emptyset$; $\mathcal{P} := \emptyset$;
$\pi_u :=$ computePPR $(G, u, \alpha)$;
$G_L :=$ localSubgraph $(G, \pi_u, k)$;
$\mathcal{C} :=$ detectCommunity $(G_L)$;
**for** each edge $e(u, v) \in G_L$ **do**
  $\mathcal{P} := \mathcal{P} \cup \{(e, \lambda_1 \cdot \pi_u(v))\}$;
**for** each community $C_i \in \mathcal{C}$ **do**
  $I_{comm}(u, C_i) := \frac{1}{|C_i|} \sum_{v \in C_i} \pi_u(v)$;
  $c_i := \min\left(\sum_{e \in \Delta_{C_i}(u)} \lambda_1 \pi_u(v),\ \lambda_2 \cdot I_{comm}(u, C_i)\right)$;
  $\mathcal{P} := \mathcal{P} \cup \{(\Delta_{C_i}(u),\ c_i)\}$;
**while** $\mathcal{M}(G \setminus \Delta, u) = t_u$ **and** $\mathcal{P} \neq \emptyset$ **do**
  $(x, s(x)) := \arg\min_{(x, s(x)) \in \mathcal{P}} s(x)$;
  $\Delta := \Delta \cup x$; $\mathcal{P} := \mathcal{P} \setminus \{(x, s(x))\}$;
**Return** $\Delta$;

**Figure 2:** Greedy **algorithm.**

**Overview.** To tackle this, we first design a *new cost function* that treats the removal of all connections between a vertex and a community as a single *macro-operation*, with a normalized cost, so that community-level and edge-level perturbations are comparable within one unified objective. Then, based on the cost function, we propose a simple heuristic method Greedy to solve MCE problem. At a high level, it (a) detects communities on the graph, (b) constructs a candidate pool mixing edge-level and community-level perturbations, each annotated with the proposed cost, and (c) greedily removes the lowest-cost candidate until the prediction flips.

We first introduce the community-aware cost function in below.

**Community-aware Cost function.** Given a graph $G$, a GNN model $\mathcal{M}$ and a target vertex $u$, our design for community-aware cost function follows two intuitions. (1) At the edge level, the cost of removing an edge should be proportional to its *influence* on the prediction $\mathcal{M}(G, u)$. That is, edges that strongly affect the prediction are more effective to perturb, and charging them by influence guides the search toward a small set of decisive edges. (2) At the community level, it is non-trivial to let a vertex disconnect with a community. If we simply summed the cost of every edge between a vertex and a community, a community-level operation would always be more expensive than any single edge and would never be selected. Hence, we normalize its cost so that a macro-operation stays comparable to, and can be preferred over, many micro edge deletions.

To measure the *influence* between two vertices (or a vertex-community pair), we adopt *Personalized PageRank* (PPR) [21] since its multi-hop diffusion from a vertex $u$ aligns with how an $L$-layer GNN propagates information within its receptive field, thus faithfully capturing each vertex's influence on the prediction $\mathcal{M}(G, u)$.

*Personalized PageRank (PPR).* Given a vertex $u \in V$, for any other vertex $v \in V$, the PPR value $\pi_u(v)$ of $v$ w.r.t $u$ is computed as below.

$$\pi_u(v) = \alpha \sum_{l=0}^{\infty} (1-\alpha)^l (\mathbf{A}^l)_{uv},$$

where $\mathbf{A}$ is the normalized adjacency matrix and $\alpha \in (0, 1]$ is the restart probability. Intuitively, the term $\mathbf{A}^l_{uv}$ corresponds to all $l$-hop paths originating from $u$ that reach vertex $v$, and the geometric coefficient $(1-\alpha)^l$ exponentially reduces the weights of farther vertices. Hence, in PPR, $\pi_u(v)$ quantitatively expresses how much vertex $v$ contributes through all possible paths starting from $u$, which is exactly the same principle as multi-hop aggregation in GNN models.

A $L$-layers GNN implicitly performs a *truncated diffusion* akin to computing the first $L$ terms of a PPR series. Therefore, PPR values describe the expected influence distribution within the GNN's receptive field. Specifically, the PPR values $\pi_u(v)$ quantify the relative influence of each vertex $v \in V$ on $u$'s message aggregation in the GNN. The higher the PPR value $\pi_u(v)$, the more important vertex $v$ to $u$.

Based on the PPR influence, we define the cost function as below.

*Edge-level.* The cost of removing an edge $e(u, v)$ is:

$$s(e) = \lambda_1 \cdot \pi_u(v),$$

where $\lambda_1 \in (0, 1]$ is a hyper-parameter to tune the effect.

*Community-level.* Given a graph $G$ and $m$ communities $\mathcal{C} = \{C_1, ..., C_i, ..., C_m\}$ identified from $G$, each vertex $v$ in $G$ is associated with multiple community labels. For a target vertex $u$, we compute a *community influence score* for $u$, denoted by $I_{comm}(u, C_i)$ as below.

$$I_{comm}(u, C_i) = \frac{1}{|C_i|} \sum_{v \in C_i} \pi_u(v),$$

where $|C_i|$ is the number of vertices in community $C_i$. This score represents how strongly $u$ is connected to community $C_i$ by the personalized diffusion process. Communities with higher $I_{comm}(u, C_i)$ contribute more to the current prediction, and thus, become potential targets for community-level counterfactual operations.

Deleting all edges linking from $u$ to vertices in community $C_i$ is aggregated as a *macro-operation* with a normalized cost in below:

$$s(\Delta_{C_i}(u)) = \min\Big( \sum_{e \in \Delta_{C_i}(u)} s(e),\ \lambda_2 \cdot I_{comm}(u, C_i) \Big),$$

where $\lambda_2$ controls the trade-off between the literal number of deleted edges and the semantic impact of leaving the community.

For a set $\Delta$ of perturbations, its total cost is defined as below:

$$s(\Delta) = \sum_{x \in \Delta} s(x).$$

By instantiating $\text{cost}(\Delta) = s(\Delta)$, we reduce MCE problem to finding

$$\Delta^* = \arg\min_{\Delta \subseteq E} s(\Delta) \text{ s.t. } \mathcal{M}(G \setminus \Delta, u) \neq t_u.$$

**Algorithm** Greedy. Based on the community-aware cost function, we propose a heuristic algorithm Greedy, as shown in Figure 2. Given a graph $G(V, E)$, a target vertex $u$, a GNN model $\mathcal{M}$ with target class $t_u$, a restart probability $\alpha$, a subgraph size $k$, and parameters $\lambda_1, \lambda_2$, it returns a perturbation set $\Delta$ such that removing $\Delta$ from $G$ flips the prediction, *i.e.,* $\mathcal{M}(G \setminus \Delta, u) \neq t_u$.

It first computes the PPR vector $\pi_u$ of the target $u$ (Line 2) and extracts a personalized local subgraph $G_L$ induced by the top-$k$ vertices with the highest PPR values (Line 3), on which communities $\mathcal{C}$ are detected (Line 4). It then constructs a candidate pool $\mathcal{P}$ that mixes both granularities: each edge $e(u, v)$ in $G_L$ is added as an edge-level candidate with cost $\lambda_1 \cdot \pi_u(v)$ (Lines 5–6), and each community $C_i$ is added as a community-level candidate that disconnects $u$ from $C_i$, with the normalized macro-cost $c_i$ derived from its influence score $I_{comm}(u, C_i)$ (Lines 7–10). During greedy selection, it repeatedly picks the lowest-cost candidate $x$ from $\mathcal{P}$, commits it to $\Delta$, and removes it (together with candidates overlapping $x$) from the pool (Lines 11–13), until the prediction of $u$ is flipped or the pool is exhausted. The final $\Delta$ forms the counterfactual explanation (Line 14).

**Example 1:** Consider a target vertex $u$ with $k = 4$, $\lambda_1 = \lambda_2 = 0.5$, whose local subgraph $G_L$ contains neighbors $v_1, v_2, v_3$ with PPR values $\pi_u(v_1) = 0.4$, $\pi_u(v_2) = 0.3$, $\pi_u(v_3) = 0.1$, and one detected community $C_1 = \{v_2, v_3\}$. Initially $\Delta = \emptyset$ and $\mathcal{P} = \emptyset$. Greedy adds three edge-level candidates with costs $s(e_1) = 0.2$, $s(e_2) = 0.15$, and $s(e_3) = 0.05$. For $C_1$, it computes $I_{comm}(u, C_1) = \frac{1}{2}(0.3 + 0.1) = 0.2$, so the macro-cost is $\min(0.15+0.05,\ 0.5\times0.2) = \min(0.2, 0.1) = 0.1$, and the community-level candidate $\Delta_{C_1}(u)$ is added with cost 0.1. In the first round, $e_3$ has the lowest cost 0.05 and is committed to $\Delta$, but $\mathcal{M}(G \setminus \Delta, u) = t_u$ still holds. Note that the min operator makes leaving the whole community $C_1$ (cost 0.1) cheaper than deleting $e_2$ alone (cost 0.15); hence in the next round $\Delta_{C_1}(u)$ is selected as a single macro-operation, flipping the prediction and yielding a compact, semantically coherent explanation. □

**Complexity.** Greedy takes $O(T_{ppr}+k+T_{comm}+k\cdot c_{gnn})$ time in total, since (a) computing the PPR vector and extracting the top-$k$ local subgraph take $O(T_{ppr})$ and $O(k)$ time; (b) community detection on $G_L$ takes $O(T_{comm})$ time; (c) building the candidate pool over the $O(k)$ vertices of $G_L$ costs $O(k)$ time; and (d) the greedy loop performs at most $O(k)$ iterations, each invoking one GNN forward pass of cost $c_{gnn}$ to check the flip condition, giving $O(k \cdot c_{gnn})$. Note that $c_{gnn} \ll |E||G|$, since the check is confined to the local subgraph $G_L$ rather than $G$, and $k \ll |V|$, so it scales to large graphs.

## 5 A TWO-STAGE FRAMEWORK: ComCE

In this section, we present an efficient two-stage method ComCE.

**Challenges**. Greedy is expensive, since it requires to restart the search from scratch over the graph $G$ for each target vertex and the number of real-GNN inferences grows with the number of edges $|E|$. However, we observe that much of this work is *target-agnostic* and repeated across queries, *e.g.,* encoding edges into a common representation, grouping role-equivalent edges, and learning which kinds of groups tend to carry counterfactual evidence do not depend on any particular $u$ and need not be redone online.

Motivated by this, we propose a *two-stage* framework ComCE, as shown in Figure 3, for improving efficiency: (1) In the *offline* phase, it performs the heavy but reusable computation once by learning the edge representation, the community structure, and a selector over communities; and (2) in the *online* phase, it executes the light query-specific work, *i.e.,* given a target vertex $u$, it extracts the communities of $u$, restricts the search to a compact community-induced domain $\mathcal{D}_u \subseteq E$, and performs a fine-grained perturbation, reserving $\mathcal{M}$ for a small number of final certification inferences. By amortizing the shared computations in the offline phase, ComCE reduces the online cost.

We elaborate the offline and online phases in Section 5.1 and Section 5.2, respectively, followed by a complexity analysis.

### 5.1 The Offline Phase

This section presents the offline phase of ComCE, which is learned once on a set of training targets and reused for all online queries.

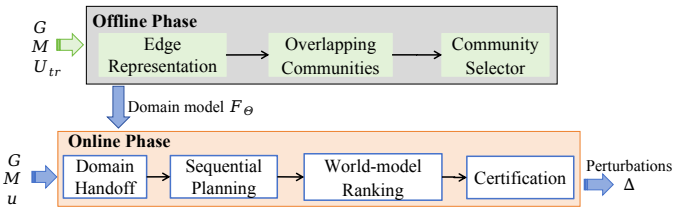



**Figure 3: The workflow of** ComCE

**Design space and challenges.** The offline phase must learn a way to tell *which communities of a target vertex are worth deleting* to flip its prediction by a GNN model $\mathcal{M}$, so that the online search can be confined to them. However, this judgement is neither purely target-agnostic nor purely target-specific due to the following two reasons: (1) the orientation of a community $C$ (*i.e.,* the role the edges in $C$ play in message passing mechanism of $\mathcal{M}$) is a global and reusable property. And (2) the *importance* of the same community differs across target vertices, *e.g.,* a community decisive for vertex $v_1$ may be irrelevant to another $v_2$ that it barely reaches. Thus, a useful selector must be reusable across different queries, yet remain sensitive to a specific target vertex at hand.

The following two straightforward designs fail to meet this requirement. *(1) Precomputing per-target answers.* One could run greedy on a pool of target vertices offline and cache the important edges of each target vertex. This merely relocates the cost of Greedy and, worse, does not generalize, *e.g.,* an unseen target vertex $v$ has no cached answer, forcing the expensive search back online. *(2) A static community score.* One could rank communities by a single importance value independent of any target vertex $v$. However, it cannot distinguish a community that is counterfactually decisive for $v$ from one that is decisive only for distant vertices, and thus, this design prunes the wrong edges.

To tackle these limitations, our ComCE learns a *model* in the offline phase that maps the *global* description of each candidate community together with a *local* target descriptor to the search domain worth exploring for a specific query vertex $v$. The model is reusable across all queries yet conditioned on $v$.

**Overview.** To do this, our offline phase produces a *domain model* $F_\Theta$ composed of three reusable components: (i) an *edge signature* $\beta_e$ that encodes the semantic coordinates of each edge $e \in E$, precomputed once and target-agnostic; (ii) a *family of relation keys* that, once a target is given, group its edges into a handful of *role-coherent overlapping communities*, turning the search unit from individual edges into communities; and (iii) a *community selector* $g_\theta$, trained with a gradient signal, that scores the counterfactual relevance of each community conditioned on the target and retains the relevant ones. At query time with a query target $u$, $F_\Theta$ chains the three components to output the search domain $\mathcal{D}_u$ of $u$ directly.

Below gives the formal problem statement of the offline phase.

- **Input:** A graph $G(V, E)$, a trained GNN model $\mathcal{M}$, and a set $\mathcal{U}_{\text{tr}}$ of training target vertices.
- **Output:** A domain model $F_\Theta$ such that, for any target $u$, $F_\Theta(u)$ infers a community-induced search domain $\mathcal{D}_u \subseteq E_u$.

Here $E_u$ denotes the edge set of the $L$-hop computation graph of $u$.

Next, we present the three components in detail.

*5.1.1 Edge Representation.* We first describe how each edge is encoded, which underlies both the community structure and the selector later. The representation is required to expose two properties

*Input:* $G(V, E)$, GNN model $\mathcal{M}$, and a set $\mathcal{U}_{\text{tr}}$ of training targets
*Output:* A domain model $F_\Theta$.

1. **for** each edge $e \in E$ **do**
2. $\quad \phi(e) := \beta_e$; // target-agnostic edge signature
3. **for** each target $v \in \mathcal{U}_{\text{tr}}$ **do**
4. $\quad \widehat{C}_v := \bigcup_r (E_v/\sim_r)$; // overlapping communities via keys $\kappa_r$
5. **while** not converged **do**
6. $\quad$ sample $v \in \mathcal{U}_{\text{tr}}$; $\mathbf{h}_v := \text{Enc}_v(v)$; // one $\mathcal{M}$ pass
7. $\quad$ **for** each $C_i \in \widehat{C}_v$ **do** $p_i := g_\theta(\widehat{C}_v, \mathbf{h}_v)$;
8. $\quad \theta := \theta - \nabla_\theta \mathcal{L}(\{p_i\}, \mathcal{M})$;
9. **Return** $F_\Theta$;

**Figure 4: The offline phase of** ComCE.

without probing $\mathcal{M}$: (i) the role-equivalence between edges, and (ii) the likelihood that an edge carries counterfactual evidence. Since both depend only on $G$ and $\mathcal{M}$, the encoding is target-agnostic.

**Semantic coordinates.** Each vertex $v$ with feature vector $\mathbf{f}_v$ is mapped by a pre-trained sentence encoder (BGE) [6] to a dense representation $\mathbf{x}_v = E_{\text{text}}(\mathbf{f}_v)$, which is projected onto $d_s$ fixed random directions $\mathbf{r}_1, \ldots, \mathbf{r}_{d_s}$ and discretized coordinate-wise by quantile bucketizers $B_j$ fitted only on the training split:

$$z_v^{(j)} = \mathbf{r}_j^\top \mathbf{x}_v, \qquad s_v^{(j)} = B_j\big(z_v^{(j)}\big) \in \{0, \ldots, m-1\}.$$

The resulting signature $\mathbf{s}_v = [\, s_v^{(1)}, \ldots, s_v^{(d_s)} \,]$ is a short vector of integer bucket ids, compact semantic coordinates following previous locality-sensitive hashing [38]. We derive an edge-level semantic representation from its endpoint embeddings and discretize it with the same projection, giving an *edge signature* $\beta_e$ used as the edge descriptor in the sequel. As $\beta_e$ depends only on $G$ and the frozen encoder, it is reusable across all targets and never recomputed online.

*5.1.2 Community Structure.* Given the edge representation, we now describe how the edges of a training target $v \in \mathcal{U}_{tr}$ are grouped into communities, so that the online search ranges over a handful of role-coherent units rather than over $|E_v|$ individual edges. The grouping must be role-coherent (*i.e.,* edges in a group exert similar counterfactual effects), yet expressive enough to separate the distinct ways an edge can matter.

To do this, we first collapse $E_v$ into a small number of *role-coherent communities* and search over these units instead. Intuitively, edges rarely matter in isolated ways, but they tend to fall into a few recurring *explanatory roles*, *e.g.,* same-relation edges one hop from the center or structurally equivalent edges in a motif. The edges sharing a role tend to exert similar counterfactual effects. We capture this along three complementary views: (i) *semantic*, (ii) *computational*, and (iii) *structural*, each of which partitions the edge set $E_v$ by a different notion of role-equivalent. Because the views disagree on what makes edges similar, a single edge can belong to several communities at once, yielding an *overlapping* community space that lets us reason about the distinct ways an edge can matter. The rest of this section formalizes the per-view keys and how they combine.

**Relation-induced communities.** Semantic coordinates alone do not explain how an edge participates in the computation of a given target $v$, so we make the role-equivalent notion of each view explicit through a canonical key. Each relation family $r$ assigns every edge $e \in E_v$ a canonical relation key $\kappa_r(e)$, and declares

$$e_i \sim_r e_j \iff \kappa_r(e_i) = \kappa_r(e_j).$$

Equality of canonical keys induces an equivalence relation $\sim_r$, whose classes $C_v^{(r)} = E_v/\sim_r$ partition $E_v$ into relation-induced communities, *i.e.,* role-equivalent edge groups that need not be topologically connected. Instantiating $\kappa_r$ recovers the three views previewed above: (i) a *semantic* view keyed by coarsened signature agreement with the center, (ii) a *computational* view keyed by hop, edge type, and center-signature agreement, and (iii) a *structural* view keyed by endpoint types, motif role, and hop. Thus, the computational key is

$$\kappa_{\text{comp}}(e) = \big(\text{hop}_v(e),\ \text{type}(e),\ b_v(e)\big),$$

where $b_v(e)$ indicates whether at least half of the edge-signature coordinates match the center signature. Thus, each induced community is a hypothesis that its edges play a similar explanatory role.

**Overlapping community space.** The three views do not merely refine one another; they intersect. Since each view partitions under a different key, communities are mutually exclusive within a view but overlap across views, *i.e.,* the same edge may simultaneously participate in a semantic, a computational, and a structural explanation. Thus, given a training target $v$, we define its overlapping explanatory community space $\widehat{C}_v$ over $E_v$ by taking the union of the per-view partitions as below:

$$\widehat{C}_v = \bigcup\nolimits_r C_v^{(r)}$$

forms the overlapping explanatory community space over $E_v$. For each edge $e$, we record the unique community it occupies in every view, $\{C_e^{(r)}\}_r$, to be used later for the per-view aggregation.

*5.1.3 Community Selector.* We learn a community selector $g_\theta$ over the overlapping communities such that $g_\theta$ must be reusable across queries yet conditioned on a specific training target $v$. To achieve both, we adopt a Set Transformer (SetT) [20] that (i) contextualizes each community against its competitors, (ii) lets the target $v$ query the community space, and (iii) scores counterfactual relevance of each community.

Intuitively, the relevance of a community is neither an edge-level nor a self-contained property; it is determined jointly by the target $v$ and by the competing communities that explain it. Thus, SetT produces the selected communities and their induced domain from a single lightweight center encoding, with no per-edge inference of GNN model $\mathcal{M}$. Below formalizes the three stages and how the selector is trained without community labels.

**Contextualizing communities.** A community should not be scored in isolation, because its explanatory role depends on where it sits relative to the target $v$ and on the competing and complementary communities around it. Thus, our SetT first encodes each community $C_i \in \widehat{C}_v$ into a token

$$\mathbf{h}_i = \text{Enc}_C\big([\,\text{content}(C_i) \,\|\, \rho_v(C_i)\,]\big) + \gamma \cdot \text{Readout}(C_i),$$

where $\text{content}(C_i)$ embeds the mode signature of $C_i$ (the per-dimension majority over its member-edge signatures) together with community statistics; $\rho_v(C_i)$ collects center-relative relational and structural descriptors and acts as a center-relative positional encoding, telling SetT where and how $C_i$ participates in the computation graph of $v$; and the learnable gate $\gamma$ controls a permutation-invariant member-edge readout $\text{Readout}(C_i)$ that preserves the fine-grained edge evidence community aggregation would otherwise erase. The tokens $\mathbf{H} = (\mathbf{h}_1, \ldots, \mathbf{h}_{|\widehat{C}_v|})$ then pass through a permutation-equivariant multi-head self-attention block,

$$\mathbf{H}' = \text{SelfAttn}(\mathbf{H}), \quad \mathbf{Q}, \mathbf{K}, \mathbf{V} \text{ all derived from } \mathbf{H}.$$

This step does not select an explanation; rather, it lets candidate explanations reason about one another, so that $\mathbf{h}'_i$ encodes $C_i$ in the context of its competing and complementary communities.

**Querying communities for the target vertex.** Having contextualized the space, we let the target $v$ query it through $K$ learnable community-query vectors $\mathbf{s}_1, \ldots, \mathbf{s}_K$, *i.e.,* trainable latent templates shared across vertices and carrying no manually assigned semantics. Each query is conditioned on the target $v$,

$$\mathbf{q}_k = W_Q^{\text{cross}}\big(\mathbf{s}_k + W_c\,\mathbf{h}_v\big),$$

where $\mathbf{h}_v$ is produced by a center encoder $\text{Enc}_v$ from $v$'s semantic signature, its pre-intervention prediction, and local-graph descriptors. The shared queries make the selector reusable, while center conditioning makes retrieval instance-specific. The contextualized communities provide keys and values $\mathbf{k}_i = W_K^{\text{cross}}\mathbf{h}'_i$ and $\mathbf{v}_i = W_V^{\text{cross}}\mathbf{h}'_i$, and each query reads via cross-attention,

$$a_{k,i} = \frac{\exp(\mathbf{q}_k^\top \mathbf{k}_i/\sqrt{d})}{\sum_j \exp(\mathbf{q}_k^\top \mathbf{k}_j/\sqrt{d})}, \quad \mathbf{p}_k = \text{LN}\big(\mathbf{q}_k + \textstyle\sum_i a_{k,i}\,\mathbf{v}_i\big).$$

Crucially, $\mathbf{h}_v$ requires only one forward pass of $\mathcal{M}$ for $v$'s pre-intervention prediction, *i.e.,* the same single inference used for certification, and no further evaluation of $\mathcal{M}$ per edge or per community, which is exactly what keeps the online cost light.

**Query–community membership and selection.** Attention determines what a query reads; membership determines what the model retains. Accordingly, a dedicated membership score rates each query–community pair, and a slot-activation logit weights the queries themselves:

$$\mu_{k,i} = \frac{(W_m^q \mathbf{p}_k)^\top (W_m^k \mathbf{h}'_i)}{\sqrt{d}}, \quad g_k = \mathbf{w}_g^\top \mathbf{p}_k.$$

Finally, we aggregate them over slots into a community-level relevance probability $p_i = \sigma\big(\log \sum \exp_k(\mu_{k,i} + g_k) + \mathbf{w}_b^\top \mathbf{h}'_i\big)$.

*Query usage.* At query time, given a query vertex $u$ and a relevance threshold $\tau \in (0, 1)$, the selected communities are

$$C_u = \{\, C_i \in \widehat{C}_u \mid p_i \geq \tau \,\} \subseteq \widehat{C}_u. \tag{2}$$

If no community passes $\tau$, we do not fall back to a single top-1 community. Instead, a soft fallback ranks the communities by $p_i$ in descending order and selects the fewest top-ranked ones. This prevents $\mathcal{D}_u$ from being compressed too far. Thus, applying the trained selector yields the selected communities $C_u$ and their edge union

$$\mathcal{D}_u = \bigcup\nolimits_{C_i \in C_u} C_i, \tag{3}$$

which we term *the community-induced search domain* of $u$. Altogether, $F_\Theta(u) = \mathcal{D}_u$, which is exactly the output required by the problem statement and is passed to the online phase.

## 5.2 The Online Phase

This section presents the online phase of ComCE. The domain model $F_\Theta$ tells *which* communities of a target vertex to weaken, but not *which specific edges* inside them form a counterfactual. Thus, the online phase first materializes the community-induced search domain $\mathcal{D}_u = F_\Theta(u)$ for the query vertex $u$, and then recovers the

missing edge-level structure within it.

**Design space and challenges.** Given a target $u$, the online phase searches $\mathcal{D}_u$ for a small edge set whose deletion flips the prediction of $\mathcal{M}$ on $u$. This is challenging due to the following three reasons. (1) *Deletion effects are non-monotone*: a set that flips $\mathcal{M}(u)$ need not still flip after one further deletion, and a set that does not flip may flip after one more; hence no monotone or greedily-committed search is sound. (2) *Calling $\mathcal{M}$ is expensive*: scoring every visited candidate with $\mathcal{M}$ would reintroduce exactly the query cost that the offline phase is designed to avoid. (3) *The domain is truncated offline*: $\mathcal{D}_u$ retains only the communities passing the relevance threshold $\tau$ (or those kept by the soft fallback), so any counterfactual lying partly outside $\mathcal{D}_u$ would be unreachable, letting completeness be decided by the selector alone.

The following two straightforward designs fail to meet these requirements. *(1) Edge-level greedy deletion* certified directly by $\mathcal{M}$ reintroduces a query count that grows with $|\mathcal{D}_u|$. *(2) Deleting the top-scored selected communities* without search is unreliable under non-monotonicity, and treating $\mathcal{D}_u$ as closed makes any edge outside it permanently unreachable.

To tackle these limitations, our ComCE searches $\mathcal{D}_u$ with a learned Dreamer-style planner [44] whose expensive and cheap operations are strictly separated. Specifically, all candidate generation and ranking are performed by cheap learned components, while the real GNN $\mathcal{M}$ is reserved for certifying a short list of candidates. In addition, the search domain can grow under a separate budget $B_{\text{ext}}$ when $\mathcal{D}_u$ proves too tight.

Below gives the formal problem statement of the online phase.

- **Input:** A graph $G(V, E)$, a GNN model $\mathcal{M}$, a query target $u$, the domain model $F_\Theta$ with relevance threshold $\tau$, a world model $F_\omega$, a planner $\pi_\xi$, a ranker $S$, budgets $B, Q, Q_{\text{clean}}, B_{\text{ext}}, N_{\text{roll}}, \ell$, a de-duplication threshold $\tau_{\text{dup}}$, a failure threshold $\tau_{\text{low}}$, and a failure limit $f_{\max}$.
- **Output:** A certified set $\Delta^* \subseteq \mathcal{D}_u^+$ with $\mathcal{M}(G - \Delta^*, u) \neq t_u$ that is locally minimal within budget, or failure, where $\mathcal{D}_u \subseteq \mathcal{D}_u^+ \subseteq E_u$ and $|\mathcal{D}_u^+| \leq |\mathcal{D}_u| + B_{\text{ext}}$.

**Algorithm.** Figure 5 gives the pseudocode of the online phase. Given a query target $u$, it runs in four steps.

*(1) Domain handoff (lines 1–4).* It first runs the GNN $\mathcal{M}$ once on the original graph to obtain the current label $t_u$ of $u$, and encodes $u$ into a short vector $\mathbf{h}_u$. This is the only call to $\mathcal{M}$ before certification. The selector then scores every community of $u$, keeps the ones whose scores reach $\tau$, and takes all of their edges as the search domain $\mathcal{D}_u$. Finally, it sets the current domain $\mathcal{D}_u^+$ to $\mathcal{D}_u$, starts with an empty candidate pool $\mathcal{P}_u$, and sets the failure counter $f$ to 0 (Section 5.2.1).

*(2) Planning and domain expansion (lines 5–9).* It then runs $N_{\text{roll}}$ rollouts. In each rollout, the planner deletes edges of the current domain one at a time, and stops when it chooses to or once $B$ edges are deleted. Every intermediate deletion set along the way is added to $\mathcal{P}_u$ as a candidate. For each candidate, the world model estimates how likely it is to flip the prediction. If none of the candidates of a rollout looks promising, the rollout counts as failed. After $f_{\max}$ failed rollouts in a row, the domain is taken to be too tight, and Expand adds one more unselected community to it, as long as the budget $B_{\text{ext}}$ allows. No step here calls $\mathcal{M}$ (Sections 5.2.2–5.2.4).

---

*Input:* $G$, $\mathcal{M}$, target $u$, $F_\Theta$ (with $\tau$), $F_\omega$, $\pi_\xi$, $S$;
budgets $B, Q, Q_{\text{clean}}, B_{\text{ext}}, N_{\text{roll}}, \ell$; $\tau_{\text{dup}}$, $\tau_{\text{low}}$, $f_{\max}$.
*Output:* certified $\Delta^* \subseteq \mathcal{D}_u^+$, or failure.

1. $t_u \leftarrow \mathcal{M}(G, u)$; $\mathbf{h}_u \leftarrow \text{Enc}_v(u)$; *// the only handoff $\mathcal{M}$ pass*
2. $\{p_i\} \leftarrow g_\theta(\widehat{\mathcal{C}}_u, \mathbf{h}_u)$;
3. $\mathcal{C}_u \leftarrow \text{Select}(\{p_i\}, \tau)$; $\mathcal{D}_u \leftarrow \bigcup_{C_i \in \mathcal{C}_u} C_i$;
4. $\mathcal{C}_u^+ \leftarrow \mathcal{C}_u$; $\mathcal{D}_u^+ \leftarrow \mathcal{D}_u$; $\mathcal{P}_u \leftarrow \emptyset$; $f \leftarrow 0$;
*// sequential planning + world-model ranking (no $\mathcal{M}$ call)*
5. **for** $n = 1$ **to** $N_{\text{roll}}$ **do**
6. $\{\Delta_t^{(n)}\}_{t=1}^{T_n} \leftarrow \text{Rollout}(\pi_\xi, u, \mathcal{D}_u^+; B)$; $\mathcal{P}_u \leftarrow \mathcal{P}_u \cup \{\Delta_t^{(n)}\}_{t=1}^{T_n}$;
7. $f \leftarrow f + 1$ if $\max_t \hat{p}_\omega(u, \Delta_t^{(n)}) < \tau_{\text{low}}$, and $f \leftarrow 0$ otherwise;
8. **if** $f \geq f_{\max}$ **then**
9. $\text{Expand}(\mathcal{D}_u^+, \mathcal{C}_u^+; B_{\text{ext}})$; $f \leftarrow 0$;
10. $\mathcal{P}_u \leftarrow \text{RankDedup}(\mathcal{P}_u, S, \tau_{\text{dup}})$;
*// budgeted real-GNN certification*
11. $\Delta_{\text{best}} \leftarrow \text{Certify}(\mathcal{P}_u; Q, \ell)$;
12. **if** $\Delta_{\text{best}} \neq \emptyset$ **then**
13. $\Delta^* \leftarrow \text{BoundedPrune}(\Delta_{\text{best}}; Q_{\text{clean}})$;
14. push $\Delta^*$ into $\mathcal{W}_u$; **return** $\Delta^*$;
15. **return** failure.

---

**Figure 5: The online phase of ComCE.**

*(3) Ranking and de-duplication (line 10).* All candidates in $\mathcal{P}_u$ are sorted by the ranker $S$, from the most to the least likely to flip. A candidate that is too similar to a higher-ranked one is dropped, so that the real GNN is not spent on near-copies (Section 5.2.5).

*(4) Certification and cleanup (lines 11–15).* Finally, the real GNN checks the top candidates one by one, at most $Q$ of them. Once a candidate flips the prediction, it checks up to $\ell$ more candidates and keeps the smallest one that flips. BoundedPrune then tries to restore its edges one at a time, and keeps a restoration only if the prediction still flips, using at most $Q_{\text{clean}}$ further calls. The resulting set is stored as a new witness and returned as $\Delta^*$. If no candidate flips, the algorithm returns failure (Section 5.2.5).

Next, we present each technique in detail.

*5.2.1 Domain Handoff.* The online phase opens by evaluating the domain model $F_\Theta$ on $u$. A single inference $t_u = \mathcal{M}(G, u)$ yields both the pre-intervention label and the center encoding $\mathbf{h}_u = \text{Enc}_v(u)$; this is the only baseline query, counted as $q_0$, and it is reused at certification. The communities $\widehat{\mathcal{C}}_u$ of $u$ are built from the relation keys $\{\kappa_r\}$, the selector $g_\theta$ scores each $C_i \in \widehat{\mathcal{C}}_u$ by $p_i$, and the selection rule of Section 5.1.3 returns the selected communities $\mathcal{C}_u$ and the domain $\mathcal{D}_u$ by Equation (3). Neither the scoring nor the selection calls $\mathcal{M}$. Owing to the soft fallback, $\mathcal{D}_u$ is never empty, so the search always starts from a non-trivial domain. Besides $\mathcal{D}_u$, we retain the scores $\{p_i\}$ and the unselected communities $\widehat{\mathcal{C}}_u \setminus \mathcal{C}_u$ (see more in Section 5.2.2).

*5.2.2 Sequential Planning.* With the domain in place, we cast edge-level search as sequential deletion, since the non-monotone landscape rules out committing to edges greedily and instead calls for exploring and *retaining* many intermediate interventions for later comparison.

**Deletion as a Markov decision process.** Edge-level search is a Markov decision process whose state exposes the center $u$, the expandable domain $\mathcal{D}_u^+$ ($\mathcal{D}_u^+ = \mathcal{D}_u$ initially), and the current deletion set $\Delta_t \subseteq \mathcal{D}_u^+$ ($\Delta_0 = \emptyset$), with actions

$$\mathcal{A}_t = \{\textsc{Add}(e) \mid e \in \mathcal{D}_u^+ \setminus \Delta_t\} \cup \{\textsc{Stop}\}.$$

$\textsc{Add}(e)$ yields $\Delta_{t+1} = \Delta_t \cup \{e\}$, and $\textsc{Stop}$ terminates the rollout, emitting $\Delta_t$ as a candidate intervention. An explanation budget $B$ bounds $|\Delta_t| \le B$ and forces termination when reached. Note that $B$ limits the size of a candidate explanation, whereas $B_{\text{ext}}$ limits the growth of the domain.

**Retaining intermediate prefixes.** A planner policy $\pi_\xi(\cdot \mid u, \Delta_t, \mathcal{D}_u^+)$ is sampled for $N_{\text{roll}}$ stochastic rollouts. Because deletion effects are non-monotone, all visited non-empty prefixes are retained for comparison rather than only terminal sets:

$$\mathcal{P}_u = \bigcup\nolimits_{n=1}^{N_{\text{roll}}} \{\Delta_t^{(n)} \mid 1 \le t \le T_n\},$$

where $\Delta_t^{(n)}$ is the deletion set after $t$ $\textsc{Add}$ steps of the $n$-th rollout and $T_n$ is its length. This turns a single trajectory into many comparable candidates.

**Budget-bounded escape hatch.** The selected domain $\mathcal{D}_u$ may miss some edges that a counterfactual needs. If $\mathcal{D}_u$ were fixed, such counterfactuals could never be found. Thus, we allow the domain to grow when the search gets stuck. Denote by $\mathcal{D}_u^+$ the current domain and $\mathcal{C}_u^+$ the communities it contains.

*When to expand?* We call a rollout *failed* if the world model (Section 5.2.3) rates all of its prefixes below a threshold $\tau_{\text{low}}$, *i.e.,* none of them is likely to flip $\mathcal{M}(u)$. If $f_{\max}$ rollouts fail in a row, we take this as a sign that the current domain is too tight.

*What to add?* We then add one more community: among the communities not yet in the domain, we pick the one with the highest selector score $p_i$, add all of its edges to $\mathcal{D}_u^+$, and reset the failure count.

*How much to add?* The total number of edges added beyond $\mathcal{D}_u$ is capped by $B_{\text{ext}}$. Since communities overlap, only the edges not already in $\mathcal{D}_u^+$ count against this budget. A community that brings no new edges, or would exceed the remaining budget, is skipped. If no community fits, the domain stays unchanged.

As a result, the domain only grows, never shrinks, and never exceeds $|\mathcal{D}_u| + B_{\text{ext}}$ edges. In short, the selector decides which edges are searched *first*, but never permanently rules out the others.

*5.2.3 Set-level World Model.* The planner produces many candidates, and checking each of them with the real GNN $\mathcal{M}$ would bring back the high query cost. To avoid this, we train a small *world model* $F_\omega$ that cheaply predicts whether deleting a candidate set $\Delta$ would flip $\mathcal{M}(u)$, without running $\mathcal{M}$.

**Scoring a candidate set.** $F_\omega$ scores a candidate $\Delta$ in two steps. It first encodes each edge $e \in \Delta$ and pools these encodings into one vector, and then combines this vector with a short summary of the target to output a flip probability:

$$\mathbf{h}_\Delta = \text{Pool}\big(\{\text{Enc}_\omega(\mathbf{x}_e) \mid e \in \Delta\}\big), \quad \hat{p}_\omega(u, \Delta) = F_\omega(\mathbf{h}_\Delta, \mathbf{c}_{u,\Delta}) \in [0, 1].$$

Here, $\mathbf{x}_e$ describes edge $e$ by its edge signature $\beta_e$ together with its relation and structure features (*e.g.,* its hop and edge type relative to $u$), and $\mathbf{c}_{u,\Delta}$ describes the context, *e.g.,* the size of $\Delta$ and how it covers the domain of $u$. Intuitively, the same set of edges should always receive the same score. Thus, we use a pooling operator because the order in which edges are deleted does not matter.

**Training.** To train $F_\omega$, we take deletion sets on the training targets in $\mathcal{U}_{\text{tr}}$, run the real GNN once on each of them offline, and record whether the prediction flips:

$$y_{u,\Delta} = \mathbb{1}\big[\mathcal{M}(G - \Delta, u) \ne t_u\big], \qquad t_u = \mathcal{M}(G, u).$$

$F_\omega$ is then fit to these labels with the binary cross-entropy loss [28].

*5.2.4 Planner Learning.* With a cheap scorer in place, we train the planner to propose small, flip-inducing, and diverse candidates. Like $g_\theta$ and $F_\omega$, the planner is trained once on $\mathcal{U}_{\text{tr}}$ and reused online; for brevity, we write the target as $u$ throughout.

**Reward.** The planner $\pi_\xi$ receives, for the terminal deletion set $\Delta$ of each rollout, the reward

$$R(u, \Delta \mid \mathcal{H}) = \lambda_{\text{flip}}\, \hat{p}_\omega(u, \Delta) - \lambda_{\text{size}} \frac{|\Delta|}{B} - \lambda_{\text{div}}\big(1 - \text{nov}(\Delta \mid \mathcal{H})\big),$$

where $\mathcal{H}$ holds the terminal sets of earlier rollouts for the same center in the batch and, with Jaccard similarity $J$, $\text{nov}(\Delta \mid \mathcal{H}) = 1 - \max_{\Delta' \in \mathcal{H}} J(\Delta, \Delta')$ (novelty 1 when $\mathcal{H}$ is empty; for an empty terminal set, $\hat{p}_\omega(u, \emptyset) = 0$ and $J(\emptyset, \emptyset) = 1$). The three terms trade off flip probability, explanation size, and diversity across the pool.

*5.2.5 Real-GNN Certification.* Finally, only a de-duplicated shortlist is verified by $\mathcal{M}$ under a query budget that is independent of the graph size, keeping the number of real-GNN calls per target at $O(1)$.

**Ranking and de-duplication.** After policy training, a copy of the world model is adapted offline to real-GNN-labeled rollout candidates, yielding a flip logit $z_{\bar{\omega}}(u, \Delta)$; the final ranking score adds a learned linear residual correction,

$$S(u, \Delta) = z_{\bar{\omega}}(u, \Delta) + \mathbf{w}_r^\top \boldsymbol{\psi}(u, \Delta) + b_r,$$

where $\boldsymbol{\psi}$ comprises pooled center-relative set descriptors. Candidates in $\mathcal{P}_u$ are ordered by decreasing $S$ and then de-duplicated by Jaccard-greedy pruning, *i.e.,* a candidate is skipped if $J > \tau_{\text{dup}}$ with any retained one, so that the budget is spent on distinct candidates.

**Budgeted certification and cleanup.** At most $Q$ survivors are tested in order against the same $t_u$ obtained at handoff, using the frozen GNN and the original graph context. Instead of first-hit termination, on a verified flip Certify continues up to $\ell$ further tests within the budget $Q$ and keeps the smallest verified $|\Delta|$. It then runs BoundedPrune, *i.e.,* one pass over the deletion set in increasing singleton score $S(u, \{e\})$, testing edge restorations until $Q_{\text{clean}}$ further queries are used or one edge remains, and accepting a restoration only if $\mathcal{M}$ still flips.

## 5.3 Theoretical Analysis

We analyze ComCE along two axes that reflect its two-stage design: its *cost*, dominated by the number of real-GNN calls (Section 5.3.1), and its *completeness*, *i.e.,* which counterfactuals the online search can in principle reach (Section 5.3.2). Throughout, we distinguish *expensive* real-GNN forward passes from *cheap* surrogate evaluations (the selector, world model, and planner). ComCE is designed to exploit exactly this split.

*5.3.1 Time Complexity.* Let $T_{\mathcal{M}}$ denote the cost of one GNN forward pass over the $L$-hop receptive field of $u$, $n = |E_u|$ the number of candidate edges, $m = |\widehat{\mathcal{C}}_u|$ the number of communities (with $m \le n$, and typically $m \ll n$), $d$ the attention width, $R$ the number of relation views, and $K$ the number of query slots. Let $C_\pi$ and $C_\omega$ denote

the per-call cost of the policy and the world model, respectively.

**Offline cost is amortized.** All heavy learning is done once and reused across all online queries. This covers edge encoding, community construction, collecting real-GNN flip labels on $\mathcal{U}_{\text{tr}}$, and training the selector, world model, and planner. The periodic fine-tuning of $F_\omega$ (Section 5.2.3) reuses labels already produced by certification. It therefore issues no extra real-GNN call and is likewise amortized over many targets. We thus focus on the per-target online cost.

**Selector pass.** Community tokenization costs $O(n)$; self-attention over the $m$ community tokens costs $O(m^2 d)$; cross-attention with $K$ slots costs $O(Kmd)$; and per-view aggregation costs $O(nR)$. A single selector pass therefore costs

$$T_{\text{sel}} = O\big(m^2 d + Kmd + nR\big),$$

with the soft fallback adding at most one $O(m \log m)$ sort. Crucially, $T_{\text{sel}}$ contains *no* real-GNN call.

**Online search.** Sampling $N_{\text{roll}}$ rollouts of at most $B$ steps costs $O\big(N_{\text{roll}} B (C_\pi + C_\omega)\big)$, since each step takes one policy pass and one world-model evaluation. This stage issues *no* real-GNN call, and its cost already covers the world-model scores used to decide whether a rollout has failed. The domain is expanded only after $f_{\max}$ consecutive failures, and each expansion admits one new community with at least one new edge. Hence the number of expansions is at most

$$N_{\text{exp}} \le \min\big\{\lfloor N_{\text{roll}}/f_{\max} \rfloor,\ |\widehat{C}_u \setminus C_u|,\ B_{\text{ext}}\big\}.$$

Each expansion counts the new edges of the outside communities and picks the best feasible one in $O(nR)$ time, again without calling $\mathcal{M}$. Ranking costs $O(|\mathcal{P}_u| \log |\mathcal{P}_u|)$ and Jaccard-greedy de-duplication costs $O(Q|\mathcal{P}_u|)$.

Certification is the *only* stage that calls the real GNN. It performs at most $Q$ flip checks and $Q_{\text{clean}}$ pruning checks, costing $O\big((Q + Q_{\text{clean}})\, T_{\mathcal{M}}\big)$. Combining the stages, the overall online time is obtained.

In practice the cost is dominated by the number of real-GNN calls. We next bound this number independently of the graph size.

Proposition 2 (Constant real-GNN query cost). *For any target $u$, the number of real-GNN forward passes issued by the online phase of* ComCE *satisfies*

$$N_{\mathcal{M}}(u) \ \le\ q_0 + Q + Q_{\text{clean}},$$

*where $q_0 = 1$ is the single baseline inference at handoff (reused at certification), and $Q, Q_{\text{clean}}$ are user-set budgets. Since $q_0, Q, Q_{\text{clean}}$ do not depend on $n = |E_u|$ or $|E|$, we have $N_{\mathcal{M}}(u) = O(1)$.*

**Proof:** The handoff step performs exactly one inference $t_u = \mathcal{M}(G, u)$. Its center encoding is cached and reused, contributing $q_0 = 1$. Sequential planning, world-model scoring, domain expansion, ranking, and de-duplication never call $\mathcal{M}$. Fine-tuning $F_\omega$ only reuses labels from certification. Certification tests at most $Q$ candidates, and BoundedPrune issues at most $Q_{\text{clean}}$ additional queries. Summing these disjoint contributions gives the bound. None of them depends on graph size, so $N_{\mathcal{M}}(u) = O(1)$. □

Proposition 2 is the core of the efficiency claim. It sets ComCE apart from exhaustive enumeration. It also sets it apart from Greedy, whose number of real-GNN calls grows with $|E_u|$.

*5.3.2 Completeness.* We separate guarantees that hold *unconditionally* from those that hold *under a sufficient budget*. We also show how the expandable domain closes the completeness gap that a fixed domain would otherwise create.

**Soundness.** Every returned set $\Delta^*$ passes the real-GNN check $\mathcal{M}(G - \Delta^*, u) \neq t_u$ (Figure 5). Besides, BoundedPrune accepts a restoration only if the flip is preserved. Hence any solution ComCE returns is a genuine counterfactual, *regardless* of the quality of the selector, world model, or planner. These surrogates affect only *which* counterfactuals are found, never *whether* a returned set is valid.

**Lemma 3:** *[Monotone and bounded domain] Assume the communities in $\widehat{C}_u$ together cover $E_u$. Under the escape hatch, the domain only grows, stays inside $E_u$, and respects the budget:*

$$\mathcal{D}_u = \mathcal{D}_u^{(0)} \subseteq \mathcal{D}_u^{(1)} \subseteq \cdots \subseteq E_u, \qquad |\mathcal{D}_u^{(k)}| \le |\mathcal{D}_u| + B_{\text{ext}} \ \text{for all } k.$$

*Moreover, if $B_{\text{ext}} \ge |E_u \setminus \mathcal{D}_u|$ and expansion is triggered at least $|\widehat{C}_u \setminus C_u|$ times, then $\mathcal{D}_u^+ = E_u$.* □

**Proof:** Each expansion adds the edges of one community $C_i \subseteq E_u$ and removes none. The domains are therefore nested and stay inside $E_u$. Only new edges are charged, and a community is admitted only if its new edges fit the remaining budget. So at most $B_{\text{ext}}$ edges are ever added.

Now let $B_{\text{ext}} \ge |E_u \setminus \mathcal{D}_u|$. The remaining budget is then at least $|E_u \setminus \mathcal{D}_u^+| \ge |C_i \setminus \mathcal{D}_u^+|$ for every $C_i$. Hence every community that brings new edges is feasible. While $\mathcal{D}_u^+ \neq E_u$, the covering assumption implies that some uncovered edge lies in a community outside $C_u^+$, so each trigger admits a new community. There are at most $|\widehat{C}_u \setminus C_u|$ such communities, which gives $\mathcal{D}_u^+ = E_u$. □

Lemma 3 turns completeness from a fragile single-point decision into a property *controlled by a budget*. We formalize this next.

Proposition 4 (Budget-bounded completeness). *Suppose a counterfactual exists for $u$. Edges outside the $L$-hop receptive field cannot affect $\mathcal{M}(u)$, so we may take a minimal one, $\Delta^* \subseteq E_u$, with $|\Delta^*| \le B$. Assume that the policy gives every in-domain edge a positive probability, and that every prefix of a sampled rollout is eligible to enter $\mathcal{P}_u$. Suppose further that:*

*(i) $\Delta^* \subseteq \mathcal{D}_u^+$ at some point of the search;*
*(ii) enough rollouts are drawn after that point; and*
*(iii) among the at most $Q$ candidates passed to certification after ranking and de-duplication, at least one is a counterfactual.*

*Then* ComCE *returns a certified counterfactual, not necessarily $\Delta^*$. Condition (i) holds trivially if $\Delta^* \subseteq \mathcal{D}_u$. Otherwise, it holds whenever the conditions of Lemma 3 for $\mathcal{D}_u^+ = E_u$ are met.*

**Proof:** By (i), every edge of $\Delta^*$ lies in the domain and is sampled with positive probability. Since $|\Delta^*| \le B$, a single rollout of at most $B$ steps can therefore pick exactly the edges of $\Delta^*$ in its first $|\Delta^*|$ steps, again with positive probability. That prefix equals $\Delta^*$. By (ii), such a rollout eventually occurs, and $\Delta^*$ becomes eligible for $\mathcal{P}_u$. By (iii), the real GNN checks a counterfactual and confirms its flip. Soundness then guarantees that the returned set is valid. □

## 6 EXPERIMENTS

This section evaluates the effectiveness and efficiency of ComCE.

| Dataset | $\|V\|$ | $\|E\|$ | #attribute | #vertex labels | #classes |
|---|---|---|---|---|---|
| Book | 2.37M | 225M | 7 | 4 | 10 |
| Claim | 702K | 1.7M | 41 | 4 | 2 |
| Computer | 3.8M | 10.4M | 4 | 2 | 34 |

**Table 1: Dataset statistics ($K = 10^3$ and $M = 10^6$)**

*Datasets.* We used three real-life graphs $G$, as shown in Table 1: (1) Book [1], a large-scale bipartite user-book interaction network with multiple user behaviors; (2) Claim [51], a medical claim network dataset designed for insurance fraud detection; and (3) Computer [52], a heterogeneous academic network in computer science.

For initial feature vectors, we adopt a dataset-specific encoding scheme according to the nature of the attributes. For Book and Computer, both with rich textual content, we encode the raw text of each vertex into a dense embedding using the pre-trained *BGE* encoder [6]. For Claim with numerical attributes, we instead adopt a per-type linear projection. Unless otherwise stated, all encoders produce feature vectors of dimension $d$.

*GNNs.* Following [48–50], we selected three GNN classifiers on heterogeneous graphs: (1) GCN [19], a standard message-passing graph convolutional network; (2) GraphSAGE [13], which effectively captures graph structural information for inductive prediction and (3) HGT [15], which designs a vertex- and edge-type dependent attention mechanism. Unless stated explicitly, we used GCN by default. All GNNs are trained with 3-layers.

*Baselines.* We tested six baselines: (1) CFExp (*i.e.,* CF-GNNExplainer) [24], which learns a sparse binary edge-deletion mask by minimizing an $L_1$-regularized loss that flips the target prediction; (2) CLEAR [27], which employs a graph variational autoencoder to generate counterfactual graphs while promoting causality and generalization across instances; (3) InduCE [40], which frames counterfactual generation as a reinforcement-learning policy that sequentially perturbs edges and generalizes inductively to unseen nodes; (4) GVEX [7], which adopts a two-tier structure, combining global graph patterns with local subgraph explanations; (5) CFGraph [2], which derives counterfactuals by adding or removing edges and aggregates them into global explanations; and (6) ATEX-CF [53], which is attack-informed and unifies adversarial edge additions with edge deletions to produce counterfactuals under a fixed perturbation budget.

The first three produce instance-level counterfactuals via optimization, a graph variational autoencoder, and reinforcement learning, respectively; GVEX and CFGraph further aggregate edge-level perturbations into global explanations, while ATEX-CF is a recent attack-informed explainer. Unlike ComCE, all of them issue a graph-size, *i.e.,* dependent number of real-GNN queries per target (see more in our Exp-2 later).

We adapted all baselines to vertex classification on heterogeneous graphs where vertices carry various attributes, and optimized all counterfactual methods to minimize perturbations.

*Environment.* Experiments were conducted on a machine powered by 256GB RAM, 32 processors with Intel(R) Xeon(R) Gold 5320 CPU @2.20GHz and a NVIDIA Tesla V100 GPU with 32 GB memory. We run each experiment 3 times and report the average. For limited space, representative results are reported on specific datasets/models.

*Evaluation metrics.* We used two widely-adopted metrics [48–50] for local explanations, together with a budget-aware variant of fidelity and two efficiency metrics. Each metric was evaluated on $N_{\text{test}}$ randomly selected testing vertices. We set $N_{\text{test}} = 1000$.

(1) *Fidelity.* Since a counterfactual perturbs the input graph for a prediction switch at $u$, the counterfactual fidelity measures the ratio of successful prediction switches, defined as follows:

$$\text{fid-cf} = \frac{1}{N_{\text{test}}} \sum\nolimits_u \mathbb{1}(\hat{y}_u \neq y_u),$$

where $\hat{y}_u$ (resp. $y_u$) is the model prediction on the perturbed $G - \Delta$ (resp. the original $G$), and $\mathbb{1}(\cdot)$ returns 1 if $\hat{y}_u \neq y_u$.

(2) *Budgeted fidelity.* In practice, the real GNN $\mathcal{M}$ can only be invoked a limited number of times per target. To measure how well each method switches predictions under such a constraint, we fix a budget of $T$ real-GNN queries per target, and define:

$$\text{fid@}T = \frac{1}{N_{\text{test}}} \sum\nolimits_u \mathbb{1}(\hat{y}_u^T \neq y_u),$$

where $\hat{y}_u^T$ is the model prediction on $G - \Delta_u^T$, and $\Delta_u^T$ is the perturbation returned by a method after it has issued at most $T$ real-GNN queries for $u$. A method is terminated once its budget is exhausted, and its current perturbation is taken as $\Delta_u^T$ ($\Delta_u^T = \emptyset$ if none); the final evaluation of $\mathcal{M}$ on $G - \Delta_u^T$ is not counted. Every forward pass of $\mathcal{M}$ issued online for $u$ is counted, including those inside gradient-based optimization and RL rollouts, while offline training queries are excluded for all methods.

(3) *Sparsity.* It measures how minimal the edge perturbations are to swap model predictions, defined as:

$$\text{spars} = \frac{1}{N_{\text{test}}} \sum\nolimits_u \frac{\#\text{ptb_edges}_u}{\#\text{edges}_u},$$

where $\#\text{ptb_edges}_u$ (resp. $\#\text{edges}_u$) is the number of perturbed edges (resp. edges in $L$-hop neighborhood graph) of $u$.

(4) *Number of real-GNN queries.* To quantify the online cost that dominates efficiency, we report the average number of real-GNN forward passes issued per target, $N_{\mathcal{M}} = \frac{1}{N_{\text{test}}} \sum_u N_{\mathcal{M}}(u)$.

(5) *Running time.* We further report the average end-to-end online latency per target (in milliseconds), and, for ComCE, separately account the one-time offline cost that is amortized across all queries.

*Default parameters.* For Greedy, we set the restart probability $\alpha = 0.15$, the local subgraph size $k = 128$, $\lambda_1 = 0.5$, and $\lambda_2 = 0.5$, and detect communities on $G_L$ by classical Louvain algorithm [12].

For ComCE, in the offline phase, we set the feature dimension $d = 64$, the number of random projections $d_s = 8$ with 16 quantile buckets per coordinate, and use 20% of the labeled vertices as training targets $\mathcal{U}_{tr}$. The selector uses $K = 3$ query slots, 4 attention heads, and hidden width 128. In the online phase, we set the relevance threshold $\tau = 0.10$, the explanation budget $B = 10$, the rollout count $N_{\text{roll}} = 32$, the failure threshold $\tau_{\text{low}} = 0.5$ with limit $f_{\max} = 4$, the expansion budget $B_{\text{ext}} = 8$, and the de-duplication threshold $\tau_{\text{dup}} = 0.9$. For certification, we set $Q = 9$, $\ell = 2$, and $Q_{\text{clean}} = 2$.

**Experimental results**. We next report our findings.

**Exp-1: Effectiveness.** We evaluated the quality of explanations.

| Dataset | Method | GCN | | | | GraphSAGE | | | | HGT | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | fid-cf | fid@$T$ | spars | time(s) | fid-cf | fid@$T$ | spars | time(s) | fid-cf | fid@$T$ | spars | time(s) |
| Book | CFExp | 0.820 | 0.000 | $6.0\times10^{-2}$ | 10.97 | 0.845 | 0.000 | $5.4\times10^{-2}$ | 10.93 | 0.812 | 0.000 | $5.9\times10^{-2}$ | 35.17 |
| | CLEAR | 0.683 | 0.413 | $11.1\times10^{-2}$ | 55.73 | 0.610 | 0.430 | $10.6\times10^{-2}$ | 43.63 | 0.662 | 0.371 | $10.3\times10^{-2}$ | 146.09 |
| | InduCE | 0.744 | 0.397 | $5.7\times10^{-2}$ | 3.33 | 0.727 | 0.365 | $5.2\times10^{-2}$ | 3.24 | 0.728 | 0.320 | $5.8\times10^{-2}$ | 8.81 |
| | GVEX | 0.706 | 0.106 | $6.6\times10^{-2}$ | 42.57 | 0.736 | 0.094 | $6.2\times10^{-2}$ | 115.00 | 0.757 | 0.083 | $6.0\times10^{-2}$ | 107.11 |
| | CFGraph | 0.728 | 0.291 | $5.3\times10^{-2}$ | 1.62 | 0.752 | 0.333 | $4.8\times10^{-2}$ | 1.60 | 0.749 | 0.317 | $4.8\times10^{-2}$ | 5.04 |
| | ATEX-CF | 0.856 | 0.000 | $4.6\times10^{-2}$ | 3.03 | 0.892 | 0.000 | $4.0\times10^{-2}$ | 2.41 | 0.869 | 0.000 | $4.5\times10^{-2}$ | 8.71 |
| | Greedy | 0.806 | 0.443 | $12.0\times10^{-2}$ | 2.87 | 0.785 | 0.385 | $11.7\times10^{-2}$ | 2.99 | 0.761 | 0.392 | $10.5\times10^{-2}$ | 3.55 |
| | ComCE | **0.951** | **0.951** | $\mathbf{3.5\times10^{-2}}$ | **0.96** | **0.966** | **0.966** | $\mathbf{3.7\times10^{-2}}$ | **1.03** | **0.923** | **0.923** | $\mathbf{3.6\times10^{-2}}$ | **1.36** |
| Claim | CFExp | 0.813 | 0.000 | $3.0\times10^{-4}$ | 126.32 | 0.796 | 0.000 | $3.1\times10^{-4}$ | 134.04 | 0.775 | 0.000 | $3.4\times10^{-4}$ | 222.45 |
| | CLEAR | 0.754 | 0.578 | $5.5\times10^{-4}$ | 590.15 | 0.613 | 0.534 | $7.2\times10^{-4}$ | 511.46 | 0.685 | 0.428 | $6.1\times10^{-4}$ | 595.20 |
| | InduCE | 0.845 | 0.474 | $3.3\times10^{-4}$ | 29.59 | 0.831 | 0.407 | $3.5\times10^{-4}$ | 23.25 | 0.805 | 0.389 | $3.8\times10^{-4}$ | 29.38 |
| | GVEX | 0.937 | 0.142 | $3.2\times10^{-4}$ | 141.97 | 0.883 | 0.084 | $4.0\times10^{-4}$ | 98.75 | 0.873 | 0.051 | $4.1\times10^{-4}$ | 104.01 |
| | CFGraph | 0.838 | 0.594 | $3.9\times10^{-4}$ | 19.91 | 0.820 | 0.497 | $4.6\times10^{-4}$ | 28.24 | 0.790 | 0.520 | $4.7\times10^{-4}$ | 25.43 |
| | ATEX-CF | 0.891 | 0.000 | $2.4\times10^{-4}$ | 21.01 | 0.844 | 0.000 | $2.8\times10^{-4}$ | 30.63 | 0.865 | 0.000 | $2.7\times10^{-4}$ | 55.37 |
| | Greedy | 0.796 | 0.534 | $8.0\times10^{-4}$ | 4.16 | 0.802 | 0.590 | $8.1\times10^{-4}$ | 4.02 | 0.773 | 0.531 | $7.8\times10^{-4}$ | 5.58 |
| | ComCE | **0.983** | **0.983** | $\mathbf{0.9\times10^{-4}}$ | **2.28** | **0.974** | **0.974** | $\mathbf{0.9\times10^{-4}}$ | **2.65** | **0.980** | **0.980** | $\mathbf{0.8\times10^{-4}}$ | **2.94** |
| Computer | CFExp | 0.715 | 0.000 | $2.0\times10^{-2}$ | 8.37 | 0.665 | 0.000 | $2.4\times10^{-2}$ | 8.85 | 0.642 | 0.000 | $2.5\times10^{-2}$ | 21.74 |
| | CLEAR | 0.629 | 0.375 | $4.1\times10^{-2}$ | 17.42 | 0.590 | 0.366 | $4.6\times10^{-2}$ | 31.45 | 0.695 | 0.396 | $4.1\times10^{-2}$ | 50.01 |
| | InduCE | 0.625 | 0.398 | $2.9\times10^{-2}$ | 2.54 | 0.668 | 0.347 | $2.6\times10^{-2}$ | 3.11 | 0.577 | 0.309 | $3.2\times10^{-2}$ | 4.75 |
| | GVEX | 0.543 | 0.064 | $4.1\times10^{-2}$ | 58.75 | 0.548 | 0.069 | $3.9\times10^{-2}$ | 45.00 | 0.550 | 0.047 | $4.1\times10^{-2}$ | 58.99 |
| | CFGraph | 0.736 | 0.337 | $1.7\times10^{-2}$ | 2.98 | 0.735 | 0.312 | $1.9\times10^{-2}$ | 3.19 | 0.775 | 0.324 | $1.6\times10^{-2}$ | **3.48** |
| | ATEX-CF | 0.767 | 0.000 | $1.8\times10^{-2}$ | **1.95** | 0.762 | 0.000 | $2.0\times10^{-2}$ | **2.09** | 0.785 | 0.000 | $1.9\times10^{-2}$ | 6.63 |
| | Greedy | 0.738 | 0.347 | $5.2\times10^{-2}$ | 2.98 | 0.707 | 0.305 | $5.3\times10^{-2}$ | 3.91 | 0.712 | 0.312 | $5.4\times10^{-2}$ | 7.97 |
| | ComCE | **0.947** | **0.947** | $\mathbf{1.1\times10^{-2}}$ | 2.16 | **0.890** | **0.890** | $\mathbf{1.2\times10^{-2}}$ | 2.15 | **0.849** | **0.849** | $\mathbf{1.2\times10^{-2}}$ | 4.22 |

**Table 2: Effectiveness of each evaluated method (best in bold).**

*Counterfactual fidelity.* As shown in Table 2, ComCE achieves the highest fid-cf in all settings. On average, its fid-cf is 23.6%, 12.6%, 43.7%, 30.5%, 22.4%, and 34.1% higher than CFExp, ATEX-CF, CLEAR, InduCE, CFGraph, and GVEX, respectively, up to 33.8%, 23.5%, 58.9%, 51.5%, 30.6%, and 74.4%. This is because ComCE searches only over a few target-relevant communities picked by the target-conditioned selector, and its world model ranks candidate perturbations for each target before $\mathcal{M}$ is queried. In contrast, the baselines either work on individual edges or rely on relaxed objectives. CFExp and ATEX-CF optimize continuous edge masks by gradient descent, and the masks may no longer flip the prediction after discretization. CLEAR trains a single generator for all targets, so its counterfactuals are not tailored to each target. InduCE and CFGraph perturb individual edges, so their search space grows with the size of the receptive field. Although GVEX achieves reasonable fid-cf on Claim, it builds local explanations from global patterns that are not specific to the target.

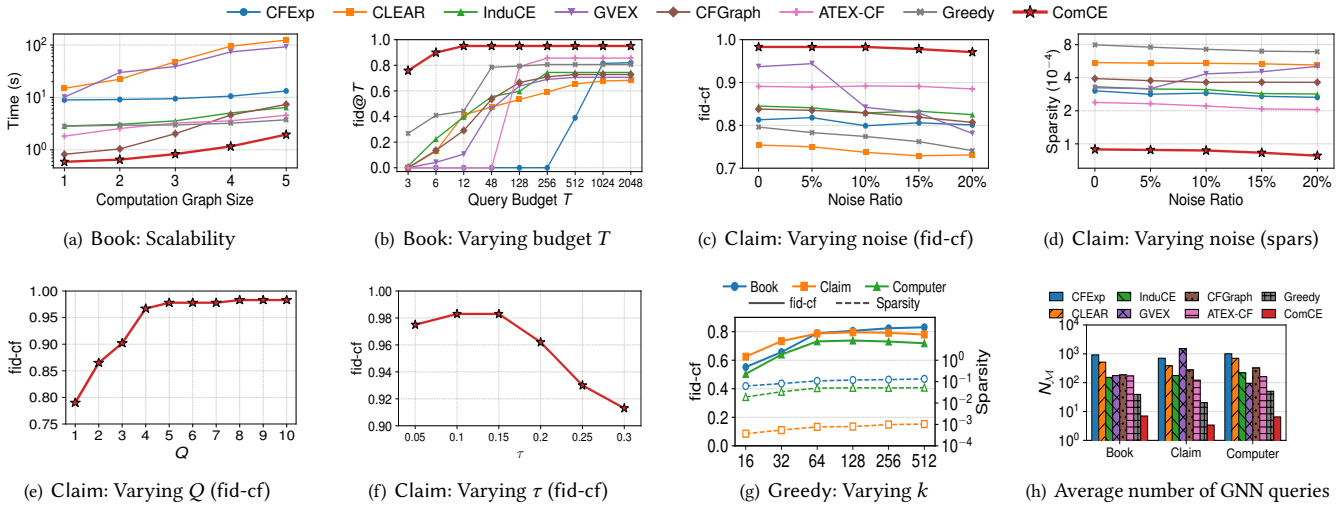
Without any training, Greedy outperforms CLEAR, InduCE, and GVEX under all three GNNs on Book and Computer, which supports the community-aware formulation itself. Greedy falls behind InduCE and GVEX on Claim; since a counterfactual removes only a tiny fraction of the receptive field, so deleting whole communities is too coarse. But ComCE closes this gap, improving over Greedy by 23.1% on average, up to 28.3%.

*Budgeted fidelity.* With 12 real-GNN queries per target vertex (*i.e.,* $T = 12$), ComCE achieves the highest fid@$T$ in all settings. Its fid@$T$ equals its fid-cf, *i.e.,* it loses no fidelity under the budget. On average, its fid@$T$ is 121.7%, 151.2%, and 154.0% higher than CLEAR, InduCE, and CFGraph, respectively, up to 152.5%, 188.4%, and 226.8%, and is 12.7× that of GVEX (up to 19.2×). Compared with the best baseline in each setting, it is 115.1% higher on average, up to 148.8%. The fid@$T$ of CFExp and ATEX-CF is 0 in all settings, since each iteration of their optimization costs one forward pass of $\mathcal{M}$, and no mask obtained within $T$ iterations flips the prediction. GVEX spends most of its queries on mining global patterns. CLEAR, InduCE, and CFGraph lose 12.9%–60.0% of their fid-cf, since every edge-level candidate must be checked by $\mathcal{M}$. ComCE also outperforms Greedy by 130.2% on average, up to 191.8%, because its world model scores candidates without querying $\mathcal{M}$.

*Sparsity.* ComCE achieves the lowest spars across all datasets. On average, its spars is 74.9%, 63.4%, 56.6%, 53.0%, 46.6%, and 40.9% lower than that of CLEAR, GVEX, InduCE, CFExp, CFGraph, and ATEX-CF, respectively, up to 87.5%, 80.5%, 79.0%, 76.5%, 83.0%, and 70.4%. CLEAR generates whole graphs and introduces changes that the target does not need. GVEX inherits edges from global patterns. InduCE never revisits earlier edits that later become redundant. CFExp, ATEX-CF, and CFGraph keep every edge whose mask passes the threshold, even if the edge is not needed for the prediction switch. Greedy has the largest spars, up to 9.8× that of ComCE, because it deletes whole communities. In contrast, ComCE keeps only target-relevant communities via the domain selector, and restores edges that are redundant for the prediction switch.

**Exp-2: Efficiency and scalability.** We next evaluate efficiency. As shown in Table 2. ComCE is efficient, taking less than 5 seconds per target, because it explores edits in the world model and calls $\mathcal{M}$ only to certify its top-ranked candidates. It is the fastest method in all settings on Book and Claim. Averaged over the nine settings, it takes 2.19 seconds to explain each target vertex; 4.6×, 29× and 103× faster than the fastest baselines CFGraph, CFExp and CLEAR, respectively. Although slightly slower than ATEX-CF and CFGraph on Computer, ComCE achieves higher fid-cf and fid@$T$.

*Number of real-GNN queries.* Figure 6(h) shows that ComCE issues at most 7 real-GNN queries per target, up to 4× (resp. 450×) fewer than Greedy (resp. other baselines). Most of the gap comes from exploration. The baselines check each step of their optimization or search against $\mathcal{M}$, whereas ComCE checks edits in the world




(a) Book: Scalability (b) Book: Varying budget $T$ (c) Claim: Varying noise (fid-cf) (d) Claim: Varying noise (spars)

(e) Claim: Varying $Q$ (fid-cf) (f) Claim: Varying $\tau$ (fid-cf) (g) Greedy: Varying $k$ (h) Average number of GNN queries

**Figure 6: Performance evaluation**

| Variant | fid-cf (↑) | fid@$T$(↑) | spars (↓) | $N_{\mathcal{M}}$(↓) | time(↓) |
|---|---|---|---|---|---|
| ComCE (full) | **0.947** | **0.947** | **0.011** | **6.5** | 2.16 |
| w/o overlap | 0.874 | 0.711 | 0.0178 | 13.6 | 3.12 |
| static-score | 0.835 | 0.653 | 0.0195 | 15.4 | 3.44 |
| w/o world model | 0.672 | 0.231 | 0.0216 | 31.8 | 5.68 |
| w/o escape hatch | 0.914 | 0.872 | 0.012 | 10.1 | **1.91** |
| w/o BoundedPrune | 0.947 | 0.947 | 0.0148 | 4.5 | 2.1 |

**Table 3: Ablation study on** Computer **(GCN).**

model and spends real queries only on its top-$Q$ candidates and on BoundedPrune. This validates the efficiency of ComCE.

*Scalability.* To evaluate scalability, we sort the 1,000 test vertices of Book in ascending order of their computation-graph sizes $|E_u|$ and split them evenly into five bins of 200 vertices each. Bin 1 (resp. bin 5) contains the vertices with the smallest (resp. largest) computation graphs. Figure 6(a) reports the average running time per target vertex in each bin. From bin 1 to bin 5, the time of ComCE rises only about 3×, and ComCE is the fastest in all bins. In contrast, the time of CLEAR, GVEX, and CFGraph grows by up to 10×, since they optimize masks over the whole computation graph. On bin 5, ComCE is about 60× faster than CLEAR.

*Varying query budget $T$.* We vary $T$ from 3 to 2048 and report fid@$T$ in Figure 6(b). As $T$ increases, ComCE converges the fastest; more specifically, it reaches its full fid-cf at $T = 12$. In contrast, Greedy needs 4× more queries to reach its plateau. ATEX-CF and CFExp find no valid counterfactual until $T \geq 128$ and $T \geq 512$, respectively. Even at $T = 2048$, no baseline exceeds 0.85 in terms of fid@$T$. This shows that ComCE is preferable when queries to $\mathcal{M}$ are limited.

**Exp-3: Ablation study.** We conducted the ablation study of ComCE in Table 3. The variants are: (i) *w/o overlap*, using a single view; (ii) *static-score*, replacing the target-conditioned selector with a static community ranking; (iii) *w/o world model*, ranking candidates by selector scores only; (iv) *w/o escape hatch*, fixing $\mathcal{D}_u$; and (v) *w/o BoundedPrune*, skipping edge restoration.

Each component contributes a distinct property. (1) *Search space.* Without overlap (resp. escape hatch), fid-cf drops by 0.073 (resp. 0.033), since counterfactuals that span multiple views (resp. lie outside $\mathcal{D}_u$) can no longer be found. (2) *Query efficiency.* Without the selector (resp. world model), $N_{\mathcal{M}}$ increases by 2.4× (resp. 4.9×) and fid@$T$ drops by 0.294 (resp. 0.716), since more candidates must be certified by $\mathcal{M}$. (3) *Conciseness.* Without BoundedPrune, spars increases by 1.3×, while the cleanup costs only 2 extra queries.

**Exp-4: Robustness to structural noise.** We randomly added or deleted a fraction of noisy edges in the receptive field and report fid-cf (resp. spars) in Figure 6(c) (resp. Figure 6(d)). As the noise ratio increases to 20%, the fid-cf of ComCE drops by at most 0.01, whereas Greedy and GVEX drop by 0.06–0.16. Meanwhile, ComCE keeps the lowest sparsity (below $10^{-4}$), about 3× lower than the best baseline. This is because random edge noise rarely changes the community structure that ComCE reasons about. This shows that community-level abstraction yields more stable explanations.

**Exp-5: Parameter sensitivity.** We varied the key parameters in our proposed methods ComCE and Greedy.

*(1) Varying $Q$.* In Figure 6(e), as $Q$ increases from 1 to 5, the fid-cf of ComCE rises from 0.79 to 0.98, and stays at about 0.98 for $Q \geq 5$. Thus, certifying the top-5 candidates is enough, *i.e.,* the world model ranks valid counterfactuals near the top.

*(2) Varying $\tau$.* In Figure 6(f), as $\tau$ increases from 0.05 to 0.3, the fid-cf of ComCE first rises from 0.975 to 0.983 at $\tau \in [0.1, 0.15]$, and then drops to 0.913 at $\tau = 0.3$. A small $\tau$ lets many irrelevant communities into $\mathcal{D}_u$, so they crowd out the valid candidates in the top-$Q$ list. A large $\tau$ filters out relevant communities, so some counterfactuals can no longer be found.

*(3) Varying $k$.* As shown in Figure 6(g), as $k$ increases from 16 to 64, the fid-cf of Greedy on Claim improves from 0.62 to 0.79, since a larger PPR-induced local subgraph covers more edges; it then plateaus for $k \geq 64$, while its sparsity keeps increasing. The same trend holds on Book and Computer.

## 7 CONCLUSION

We studied minimal counterfactual explanations for GNN-based vertex classification. We first proposed a community-aware cost function and a Greedy algorithm, and then developed ComCE, a two-stage framework that amortizes reusable offline learning (edge representation, overlapping communities, and a target-conditioned selector) so that the online search issues only $O(1)$ real-GNN calls per target. We proved its soundness and budget-controlled completeness, and empirically verified that ComCE attains the best fidelity and sparsity while keeping its query count flat as graphs grow.

# REFERENCES


[1] 2024. Goodreads Book Reviews. *https://www.kaggle.com/datasets/pypiahmad/goodreads-book-reviews1/.*

[2] Carlo Abrate and Francesco Bonchi. 2021. Counterfactual graphs for explainable classification of brain networks. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2495–2504.

[3] Mohit Bajaj, Lingyang Chu, Zi Yu Xue, Jian Pei, Lanjun Wang, Peter Cho-Ho Lam, and Yong Zhang. 2021. Robust counterfactual explanations on graph neural networks. *Advances in Neural Information Processing Systems* 34 (2021), 5644–5655.

[4] Federico Baldassarre and Hossein Azizpour. 2019. Explainability Techniques for Graph Convolutional Networks. In *ICML Workshop "Learning and Reasoning with Graph-Structured Representations"*.

[5] Ruichu Cai, Yuxuan Zhu, Xuexin Chen, Yuan Fang, Min Wu, Jie Qiao, and Zhifeng Hao. 2025. On the probability of necessity and sufficiency of explaining graph neural networks: A lower bound optimization approach. *Neural Networks* 184 (2025), 107065.

[6] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2318–2335. doi:10.18653/v1/2024.findings-acl.137

[7] Tingyang Chen, Dazhuo Qiu, Yinghui Wu, Arijit Khan, Xiangyu Ke, and Yunjun Gao. 2024. View-based explanations for graph neural networks. *Proc. ACM Manag. Data* 2, 1 (2024), 1–27.

[8] Lihang Fan, Wenfei Fan, Ping Lu, Chao Tian, and Qiang Yin. 2024. Enriching Recommendation Models with Logic Conditions. *Proc. ACM Manag. Data* (2024).

[9] Thorben Funke, Megha Khosla, Mandeep Rathee, and Avishek Anand. 2022. Z orro: Valid, sparse, and stable explanations in graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* (2022).

[10] Chen Gao, Yu Zheng, Nian Li, Yinfeng Li, Yingrong Qin, Jinghua Piao, Yuhan Quan, Jianxin Chang, Depeng Jin, Xiangnan He, and Yong Li. 2023. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems* 1, 1 (2023), 1–51.

[11] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.

[12] Sayan Ghosh, Mahantesh Halappanavar, Antonino Tumeo, Ananth Kalyanaraman, Hao Lu, Daniel Chavarria-Miranda, Arif Khan, and Assefaw Gebremedhin. 2018. Distributed louvain algorithm for graph community detection. In *2018 IEEE international parallel and distributed processing symposium (IPDPS)*. IEEE, 885–895.

[13] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. *Advances in neural information processing systems* 30 (2017).

[14] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together. In *SIGMOD*. 1429–1446.

[15] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *The Web conference 2020*. 2704–2710.

[16] Qiang Huang, Makoto Yamada, Yuan Tian, Dinesh Singh, and Yi Chang. 2022. GraphLIME: Local Interpretable Model Explanations for Graph Neural Networks. *TKDE* 35, 7 (2022), 6968–6972.

[17] Zexi Huang, Mert Kosan, Sourav Medya, Sayan Ranu, and Ambuj Singh. 2023. Global counterfactual explainer for graph neural networks. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 141–149.

[18] Di Jin, Zhizhi Yu, Pengfei Jiao, Shirui Pan, Dongxiao He, Jia Wu, Philip S Yu, and Weixiong Zhang. 2021. A survey of community detection approaches: From statistical modeling to deep learning. *TKDE* 35, 2 (2021), 1149–1170.

[19] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. *ICLR* (2017).

[20] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. 2019. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*. PMLR, 3744–3753.

[21] Dandan Lin, Raymond Chi-Wing Wong, Min Xie, and Victor Junqiu Wei. 2020. Index-free approach with theoretical guarantee for efficient random walk with restart query. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 913–924.

[22] Yifei Liu, Chao Chen, Yazheng Liu, Xi Zhang, and Sihong Xie. 2021. Multi-objective explanations of GNN predictions. In *2021 IEEE International Conference on Data Mining (ICDM)*. IEEE, 409–418.

[23] Yunchao Lance Liu, Yu Wang, Oanh Vu, Rocco Moretti, Bobby Bodenheimer, Jens Meiler, and Tyler Derr. 2023. Interpretable chirality-aware graph neural network for quantitative structure activity relationship modeling in drug discovery. In *AAAI*, Vol. 37. 14356–14364.

[24] Ana Lucic, Maartje A Ter Hoeve, Gabriele Tolomei, Maarten De Rijke, and Fabrizio Silvestri. 2022. CF-GNNExplainer: Counterfactual Explanations for Graph Neural Networks. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 4499–4511.

[25] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. *NeurIPS* 33 (2020), 19620–19631.

[26] Yice Luo, Guannan Wang, Yongchao Liu, Jiaxin Yue, Weihong Cheng, and Binjie Fei. 2023. FAF: A Risk Detection Framework on Industry-Scale Graphs. In *CIKM*. 4717–4723.

[27] Jing Ma, Ruocheng Guo, Saumitra Mishra, Aidong Zhang, and Jundong Li. 2022. Clear: Generative counterfactual explanations on graphs. *Advances in neural information processing systems* 35 (2022), 25895–25907.

[28] Anqi Mao, Mehryar Mohri, and Yutao Zhong. 2023. Cross-entropy loss functions: Theoretical analysis and applications. In *International conference on Machine learning*. pmlr, 23803–23828.

[29] Tri Minh Nguyen, Thomas P Quinn, Thin Nguyen, and Truyen Tran. 2022. Explaining black box drug target prediction through model agnostic counterfactual samples. *IEEE/ACM Transactions on Computational Biology and Bioinformatics* 20, 2 (2022), 1020–1029.

[30] Danilo Numeroso and Davide Bacciu. 2021. Meg: Generating molecular counterfactual explanations for deep graph networks. In *2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.

[31] Phillip E Pope, Soheil Kolouri, Mohammad Rostami, Charles E Martin, and Heiko Hoffmann. 2019. Explainability methods for graph convolutional neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10772–10781.

[32] Dazhuo Qiu, Mengying Wang, Arijit Khan, and Yinghui Wu. 2024. Generating Robust Counterfactual Witnesses for Graph Neural Networks. *ICDE* (2024).

[33] Susie Xi Rao, Shuai Zhang, Zhichao Han, Zitao Zhang, Wei Min, Zhiyao Chen, Yinan Shan, Yang Zhao, and Ce Zhang. 2021. xFraud: Explainable fraud transaction detection. *PVLDB* 15, 3 (2021), 427–436.

[34] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. 2021. Interpreting Graph Neural Networks for NLP with Differentiable Edge Masking. In *International Conference on Learning Representations*. https://mlanthology.org/iclr/2021/schlichtkrull2021iclr-interpreting/

[35] Mengying Sun, Sendong Zhao, Coryandar Gilvary, Olivier Elemento, Jiayu Zhou, and Fei Wang. 2020. Graph convolutional networks for computational drug development and discovery. *Briefings in bioinformatics* 21, 3 (2020), 919–935.

[36] Yi Sun, Abel Valente, Sijia Liu, and Dakuo Wang. 2021. Preserve, promote, or attack? gnn explanation via topology perturbation. *arXiv preprint arXiv:2103.13944* (2021).

[37] Juntao Tan, Shijie Geng, Zuohui Fu, Yingqiang Ge, Shuyuan Xu, Yunqi Li, and Yongfeng Zhang. 2022. Learning and evaluating graph neural network explanations based on counterfactual and factual reasoning. In *The Web Conference*. 1018–1027.

[38] Zongyuan Tan, Hongya Wang, Bo Xu, Minjie Luo, and Ming Du. 2025. Fast Locality Sensitive Hashing with Theoretical Guarantee. In *International Conference on Case-Based Reasoning*. Springer, 298–312.

[39] Berk Ustun, Alexander Spangher, and Yang Liu. 2019. Actionable recourse in linear classification. In *Proceedings of the conference on fairness, accountability, and transparency*. 10–19.

[40] Samidha Verma, Burouj Armgaan, Sourav Medya, and Sayan Ranu. 2024. InduCE: Inductive Counterfactual Explanations for Graph Neural Networks. *Transactions on Machine Learning Research* (2024).

[41] Sahil Verma, Varich Boonsanong, Minh Hoang, Keegan Hines, John Dickerson, and Chirag Shah. 2024. Counterfactual explanations and algorithmic recourses for machine learning: A review. *Comput. Surveys* 56, 12 (2024), 1–42.

[42] Sandra Wachter, Brent Mittelstadt, and Chris Russell. 2017. Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harv. JL & Tech.* 31 (2017), 841.

[43] Geemi P Wellawatte, Aditi Seshadri, and Andrew D White. 2022. Model agnostic generation of counterfactual explanations for molecules. *Chemical science* 13, 13 (2022), 3697–3705.

[44] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. 2023. Daydreamer: World models for physical robot learning. In *Conference on robot learning*. PMLR, 2226–2240.

[45] Shiwen Wu, Wentao Zhang, Fei Sun, and Bin Cui. 2020. Graph Neural Networks in Recommender Systems: A Survey. *CoRR* abs/2011.02260 (2020). arXiv:2011.02260 https://arxiv.org/abs/2011.02260

[46] Jiacheng Xiong, Zhaoping Xiong, Kaixian Chen, Hualiang Jiang, and Mingyue Zheng. 2021. Graph neural networks for automated de novo drug design. *Drug discovery today* 26, 6 (2021), 1382–1393.

[47] Makoto Yamada, Wittawat Jitkrittum, Leonid Sigal, Eric P Xing, and Masashi Sugiyama. 2014. High-dimensional feature selection by feature-wise kernelized lasso. *Neural computation* 26, 1 (2014), 185–207.

[48] Zhitao Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. GNNExplainer: Generating Explanations for Graph Neural Networks. In *NeurIPS*. 9240–9251.

[49] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2022. Explainability in graph neural networks: A taxonomic survey. *IEEE transactions on pattern analysis and machine intelligence* 45, 5 (2022), 5782–5799.

[50] Hao Yuan, Haiyang Yu, Jie Wang, Kang Li, and Shuiwang Ji. 2021. On explainability of graph neural networks via subgraph explorations. In *ICML*. PMLR, 12241–12252.

[51] Rui Zhang, Dawei Cheng, Jie Yang, Yi Ouyang, Xian Wu, Yefeng Zheng, and Changjun Jiang. 2024. Pre-trained online contrastive learning for insurance fraud detection. In *AAAI*, Vol. 38. 22511–22519.

[52] Yu Zhang, Bowen Jin, Qi Zhu, Yu Meng, and Jiawei Han. 2023. The effect of metadata on scientific literature tagging: A cross-field cross-model study. In *Proceedings of the ACM Web Conference 2023*. 1626–1637.

[53] Yu Zhang, Sean Bin Yang, Arijit Khan, and Cuneyt Gurcan Akcora. 2026. ATEX-CF: Attack-Informed Counterfactual Explanations for Graph Neural Networks. In *International Conference on Learning Representations (ICLR)*.

| Notations | Definitions |
|---|---|
| $G$, $Q[x_0, y_0]$ | graph, dual star pattern $Q[x_0, y_0] = \langle Q_x[x_0, \bar{x}], Q_y[y_0, \bar{y}]\rangle$ |
| $\mathcal{M}(x_0, y_0)$ | a GNN model that predicts how user $x_0$ likes item $y_0$ |
| $2\mathsf{WL}_L(z, z')$ | the predicate for local 2-WL test |
| $X$ | the precondition (a conjunction of logic predicates) |
| $\varphi, \Sigma$ | $\mathsf{REP}\ \varphi = Q[x_0, y_0](X \to \mathcal{M}(x_0, y_0))$, a set of REPs |
| $z, z.A, w$ | a variable in $\bar{x} \cup \bar{y}$, an attribute/feature of $z$, a vertex in $G$ |
| $\rho$ | a path $\rho = (z_0, z_1, \ldots, z_n)$ with length $\lvert\rho\rvert = n$ |
| $(u, v)$ | a pair of user and item |
| $h$ | a match of $Q$ of $\varphi$ that maps each $z$ in $Q$ to $w$ in $G$ (*i.e.,* $h(z) = w$); if $h \models \varphi$, $h(x_0) = u$ and $h(y_0) = v$, $h$ is a witness of $\varphi$ |
| $(\varphi, h)$ | an evidence (an applicable $\varphi$ and a witnesses of $\varphi$ at $(u, v)$) |
| $\mathbb{H}_{(u,v)}(\Sigma, G)$ | the set of evidences for $(u, v)$ by REPs in $\Sigma$. |
| $\mathbb{O}$ | a set of vertex features modified or edges removed from $G$ |
| $G \ominus \mathbb{O}$ | $G$ updated by the feature modification and edge removal in $\mathbb{O}$ |
| $s(\varphi, h)$ | the ranking/importance score of $(\varphi, h)$ |
| $s(\varphi), s_\varphi(h)$ | rule, witness importance $(s(\varphi, h) = s(\varphi) \cdot s_\varphi(h))$ |
| $s_\varphi(Q), s_\varphi(X)$ | pattern, precondition importance $(s_\varphi(h) = s_\varphi(Q), \cdot s_\varphi(h))$ |
| $\beta_{w.A}, d_w, \Delta_w$ | the feature importance of $w$, the degree of $w$, an indicator of $w$ |
| $P_X$, $\mathsf{Gini}(P_X)$ | a set of $(u, v)$ s.t. $\forall (u, v) \in P_X$, $\exists h \models X$, the GINI value of $P_X$ |
| $\Delta\mathsf{Gini}_{z.A}$ | the reduction of the GINI value by involving $z.A$ in $X$ |
| $\mathsf{value}(w.A)$ | the contribution of the actual $A$-attribute values of $w$ |
| $s(w), \hat{s}(z)$ | the importance score of $w$, the importance upper bound of all vertices mapped to $z$ |
| $\Psi, T_k$ | a heap of top-$k$ evidences, the $k$-th highest ranking score |
| $\hat{s}(\varphi)$ | a score upper bound for all possible witnesses $h$ of $\varphi$ |
| $C_\rho(z), C_\varphi(z)$ | a set of vertices $w$ in $G$ s.t. there exists $h$ of $\rho/\varphi$ and $h(z) = w$ |
| $\mathsf{cost}(w), \mathsf{eff}(w)$ | the cost/effectiveness of removing a vertex $w$ in $C_\varphi$ |
| $\alpha, \tau, \lambda$ | a hop-decay factor, a degree parameter, a balancing parameter |

**Table 4: Notations**

# APPENDIX

## Appendix A: Notation Table

The notations of the paper are summarized in Table 4.

## Appendix B: Missing Proofs

**Proof of Proposition ??.** We show that the following problem is NP-complete: given a graph $G$, a set $\Sigma$ of REPs, a GNN-based recommendation model $\mathcal{M}$, a pair $(u, v)$, and a positive integer $K$, whether the size of counterfactual explanations $\mathbb{O}$ is at most $K$.

For the upper bound, we use the following algorithm: first guess a set $\mathbb{O}^s$ of at most $K$ graph perturbations, and then check whether $\mathbb{O}^s$ makes no $\varphi \in \Sigma$ applicable at $(u, v)$ to witness $\mathcal{M}(u, v)$. The checking can be done in PTIME (see the argument in Section ??). Hence the algorithm is in NP, and so is the problem.

We prove the NP-hardness by reduction from the vertex cover problem [11], which is to decide, given an undirected graph $G_c = (V_c, E_c)$ and a positive integer $K_c$, whether there exists a vertex cover of size at most $K_c$, *i.e.,* a subset $V_1 \subseteq V$ with $|V_1| \leq K_c$ such that for each edge $(u, v) \in E$, either $u \in V_1$ or $v \in V_1$. Given $G_c$ and $K_c$, we construct a directed graph $G$, a set $\Sigma$ of REPs, a GNN-based model $\mathcal{M}$, a designated pair $(u, v)$ in $G$, and a positive integer $K$ such that $|\mathbb{O}| \leq K$ if and only if $G_c$ has a vertex cover of size at most $K_c$. Intuitively, (1) $G$ encodes the topological structure of $G_c$, and (2) the REPs in $\Sigma$ inspect edges in $G_c$, where each edge corresponds to an evidence of the recommendation of $\mathcal{M}$ at $(u, v)$. In this way, a vertex cover in $G_c$ corresponds to a minimal set of graph perturbations, *i.e.,* the counterfactual explanation, which make no REPs in $\Sigma$ applicable at $(u, v)$ anymore, *i.e.,* the prediction $\mathcal{M}(u, v)$ is swapped.

We define the graph $G$, the set $\Sigma$ of REPs, the GNN-based model $\mathcal{M}$, a user-item pair $(u_0, v_0)$ in $G$, and bound $K$ as follows.

(1) The graph $G = (V, E, L, F_A)$ is defined as follows: (a) $V = V_c \cup \{u_0, v_0\}$, *i.e., G* has the same set of vertices as $G_c$, plus two dummy vertices, namely $u_0$ and $v_0$. For simplicity, we use $w$ to denote a non-dummy vertex in $V$ (*i.e.,* $w \neq u_0 \neq v_0$). (b) $E = \{(w_a, \tau, w_b), (w_b, \tau, w_a) \mid (w_a, w_b) \in E_c\} \cup \{(u_0, \mathsf{likes}, v_0)\} \cup \{(u_0, \tau, w) \mid w \in V_c\}$, *i.e.,* an undirected edge $(w_a, w_b)$ in $G_c$ is encoded by two directed edges $(w_a, \tau, w_b)$ and $(w_b, \tau, w_a)$ in $G$; moreover, $G$ includes $|V_c|$ dummy edges, labeled $\tau$, from $u$ to the vertices $w$ in $V_c$ and a dummy edge from $u_0$ to $v_0$ with label likes. (c) For each $w \in V$, $w$ is labeled $\eta$ and $w$ carries an attribute $w.A = c$ for a constant $c$; *i.e.,* all vertices in $V$ except $u_0$ and $v_0$ carry the same attribute bearing the same value. (d) Vertex $u_0$ and $v_0$ are labeled $\eta_u$ and $\eta_v$, respectively, where $\eta \neq \eta_u \neq \eta_v$; moreover, $u_0$ and $v_0$ do not carry any attribute.

(2) The set $\Sigma$ consists of one REP: $\varphi = Q[x_0, y_0](X \to \mathcal{M}(x_0, y_0))$, where (a) $Q$ consists of two star patterns $Q_x$ and $Q_y$, where $Q_x$ consists of one path $\rho_x = (x_0, \tau, x_1, \tau, x_2)$, $Q_y$ consists of an isolated vertex $y_0$; (b) $x_1$ and $x_2$ are labeled $\eta$ and the two centers $x_0$ and $y_0$ are labeled $\eta_u$ and $\eta_v$, respectively; (c) $X$ is $x_1.A = c \wedge x_2.A = c$; and (d) $\mathcal{M}$ is any GNN-based recommendation model that satisfies the following assumption: if a user $u_0$ has a path to all items in a user-item interaction graph, and if $u_0$ likes a particular item $v_0$, then $\mathcal{M}$ predicts true at this user-item pair, *i.e.,* it will recommend the item $v_0$ to the user $u_0$. Since $u_0$ has a path to all vertices $w$ in $G$ (since $(u_0, \tau, w) \in E$) and $(u_0, \mathsf{likes}, v_0) \in E$, such $\mathcal{M}$ predicts true at $(u_0, v_0)$.

Observe the following: (a) Since only the label of $u_0$ (resp. $v_0$) maps the label of center $x_0$ (resp. $y_0$) of $\varphi$, all witnesses of $\varphi$ must pivot at $(u_0, v_0)$. (b) Since all edges except $(u_0, \mathsf{likes}, v_0)$ in $G$ carry the same labels and the $A$-attribute values of all vertices except $u_0$ and $v_0$ in $G$ are $c$, each edge $(w_a, \tau, w_b)$ in $G$ has an one-to-one relationship with an evidence $(\varphi, h_{(w_a, w_b)})$ in $\mathbb{H}_{(u_0, v_0)}(\Sigma, G)$; one can easily verify that $h_{(w_a, w_b)}$ is a match that maps vertices $u_0$, $w_a$, $w_b$, $v_0$ to variables $x_0, x_1, x_2, y_0$, respectively. (c) When the attribute value of one of the two vertices $w_a$ and $w_b$ in $G$ is modified to a new constant $c'$ ($c' \neq c$), the match $h_{(w_a, w_b)}$ is no longer a witness of $\varphi$ since the precondition $X$ is not satisfied; in this case, $(\varphi, h_{(w_a, w_b)})$ becomes *invalid.* (d) If a counterfactual explanation invalidate an evidence $(\varphi, h)$ in $\mathbb{H}_{(u_0, v_0)}(\Sigma, G)$ by removing an edge $e$ from $G$, we can construct another counterfactual explanation with smaller or equal size by modifying the attribute values of non-dummy vertices in $h$, *e.g.,* if we remove an edge $(w_a, \tau, w_b)$ in $G$, we can only invalidate one evidence $(\varphi, h_{(w_a, w_b)})$; however, if we modify the attribute value of $w_a$, we can invalidate the evidences $(\varphi, h_{(w_a, w)})$ for all neighbors $w$ of $w_a$ in $G$, since the predicate $x_1.A = c$ is violated. Therefore, we can assume that the graph perturbations in $\mathbb{O}$ consists of only attribute/feature modification.

(3) For the bound of the counterfactual explanation, we set $K = K_c$.

We next show the correctness of the reduction, *i.e., $G_c$* has a vertex cover of size at most $K_c$ if and only if the size of the counterfactual explanation $\mathbb{O}$ for the pair $(u_0, v_0)$ is at most $K$.

($\Rightarrow$) Assume that $G_c$ has a vertex cover of size at most $K_c$. We modify at most $K$ attribute values such that the only REP in $\Sigma$ cannot be applied at $(u_0, v_0)$, *i.e.,* the size of the counterfactual explanation $\mathbb{O}$ for $(u_0, v_0)$ is at most $K$. Let $w_1, \ldots, w_{K_c}$ be a vertex cover of $G_c$ and $c_1, \ldots, c_{K_c}$ be $K_c$ distinct constants that do not appear in $G$. Then $\mathbb{O}$ consists of $K_c$ feature modification, where the $i$-th one

modifies the $A$-attribute value of $v_i$ to be $w_i.A = c_i$ ($i \in [1, K_c]$).

We next show that the only REP $\varphi$ in $\Sigma$ is no longer applicable at $(u_0, v_0)$. For each edge $(w_a, \tau, w_b)$ in $G$, we assume *w.l.o.g.* that $(w_a, \tau, w_b) \in E$ is encoded from $(w_a, w_b) \in E_c$. Recall that each $(w_a, \tau, w_b)$ in $G$ has a one-to-one relationship with an evidence $(\varphi, h_{(w_a,w_b)})$ in $\mathbb{H}_{(u_0,v_0)}(\Sigma, G)$. Since $w_1, \ldots, w_{K_c}$ is a vertex cover, either $w_a$ or $w_b$ exists in $\{w_1, \ldots, w_{K_c}\}$, and its $A$-attribute value has been modified to a distinct new constant in $G \ominus \mathbb{O}$. Then $(\varphi, h_{(w_a,w_b)})$ becomes invalid in $\mathbb{H}_{(u_0,v_0)}(\Sigma, G \ominus \mathbb{O})$. Since all evidences becomes invalid, the only REP in $\Sigma$ is no longer applicable.

($\Leftarrow$) Assume that the set $\mathbb{O}$ consists of $K$ feature modification, *i.e.,* $f_1, \ldots, f_K$, as the counterfactual explanation that makes $\Sigma$ no longer applicable at $(u_0, v_0)$. Since neither $u_0$ nor $v_0$ carry any attribute, each $f_i$ modifies the attribute value of a non-dummy vertex $w$ in $V$. Let $w_1, \ldots, w_K$ be the vertices whose attribute values are modified. We show that $w_1, \ldots, w_K$ form a vertex cover of $G_c$. For any edge $(w_a, w_b)$ in $G_c$, since the REP $\varphi \in \Sigma$ cannot be applied at $(u_0, v_0)$, we know that the attribute value of either $w_a$ or $w_b$ is modified to a new constant in $G \ominus \mathbb{O}$. In other words, either $w_a$ or $w_b$ exists in $w_1, \ldots, w_K$ and thus, $w_1, \ldots, w_K$ form a vertex cover of $G_c$.

## Appendix C: Rule Discovery

Below we first formulate the discovery problem, and then present more details about the discovery algorithm RepsLearner.

**Criteria**. We start with two measures to evaluate the quality of REPs. Consider REP $\varphi = Q[x_0, y_0](X \to \mathcal{M}(x_0, y_0))$ and graph $G$.

*Support.* This is to measure how often a REP $\varphi$ can be applied in graph $G$. More specifically, the *support* of $\varphi$ in $G$ is defined as:

$$\mathsf{supp}(\varphi, G) = \|Q(x_0, y_0, G, X \wedge \mathcal{M}(x_0, y_0))\|.$$

Here $Q(x_0, y_0, G, X \wedge \mathcal{M}(x_0, y_0))$ is the set of pairs $(h(x_0), h(y_0))$ for all matches $h$ of $Q$ in $G$ such that $h \models X \wedge \mathcal{M}(x_0, y_0)$. Intuitively, the higher $\mathsf{supp}(\varphi, G)$ is, the more frequent $\varphi$ can be applied to $G$.

*Confidence.* It measures how strong the connection between the precondition $X$ and prediction $\mathcal{M}(x_0, y_0)$ is. The confidence of $\varphi$ in $G$ is:

$$\mathsf{conf}(\varphi, G) = \frac{\mathsf{supp}(\varphi, G)}{\|Q(x_0, y_0, G, X)\|}.$$

**REP discovery**. We formalize the discovery problem as follows.

- *Input*: A graph $G$, a GNN model $\mathcal{M}$, a support threshold $\sigma > 0$, a confidence threshold $\delta > 0$, and positive integers $\alpha_1$, $\alpha_2$ and $\alpha_3$.
- *Output*: A set $\Sigma$ of REPs pertaining to $\mathcal{M}$ such that for each $\varphi \in \Sigma$, $\mathsf{supp}(\varphi, G) \geq \sigma$, $\mathsf{conf}(\varphi, G) \geq \delta$, $X$ has at most $\alpha_1$ predicates, $Q$ has at most $\alpha_2$ paths and each path has at most $\alpha_3$ variables.

We use $\mathsf{Req}(\sigma, \delta, \alpha_1, \alpha_2, \alpha_3)$ to denote the discovery requirement. Intuitively, REPs satisfying $\mathsf{Req}(\sigma, \delta, \alpha_1, \alpha_2, \alpha_3)$ are able to simulate the outcomes of $\mathcal{M}$, and have bounded patterns/preconditions. Here $\alpha_1, \alpha_2$ and $\alpha_3$ are mostly to control the cost of rule discovery.

**Algorithm**. RepsLearner learns REPs in rounds; in each round, it has two steps: (1) pattern generation and (2) precondition mining. The precondition mining is similar to that in [8], via levelwise search. Thus, we focus on pattern generation in the following.

**Pattern generation.** Given graph $G$ and model $\mathcal{M}$, RepsLearner returns a set of star patterns in two steps. (1) It computes a subgraph $G_{\mathrm{sg}}(u, v)$ pertaining to $\mathcal{M}$ for each user-item pair $(u, v)$ in $G$, by adapting MCTS. (2) It employs random walks starting from users (resp. items) in $G_{\mathrm{sg}}(u, v)$ to get a list of paths; these paths compose star patterns $Q_x$ (resp. $Q_y$) centered at $x_0$ (resp. $y_0$).

*(1) Monte Carlo tree search (MCTS).* For each pair $(u, v)$ in $G$, if $\mathcal{M}$ predicts true at $(u, v)$, we build a search tree in which the root node is associated with the $L$-hop neighborhood graph $G_L(u, v)$ of $u$ and $v$, where $L$ is the number of layers for $\mathcal{M}$ to aggregate information from the neighbors of $u$ and $v$, and each of the other tree nodes is associated with a connected subgraph of $G_L(u, v)$. Each edge in the search tree represents a pruning action that prunes a vertex from the graph associated with its parent node, to get a new subgraph at the child node; each node can have multiple pruning actions, leading to different children nodes. The MCTS records the statistics of *visiting counts* and *rewards* to guide the exploration iteratively, so that in early iterations, the MCTS tends to select children nodes with low visit counts in order to explore different possible pruning actions. In later iterations, the MCTS tends to select children nodes that yield higher rewards, *i.e.,* more important subgraphs.

Both the MCTS rewards and the explanation selection are computed based on *the shapley values*, which were proposed from the cooperative game theory for fairly assigning a total game gain to different game players. To apply it to our settings, we use the GNN prediction of $\mathcal{M}$ as the game gain and different graph structures as players. After searching for several iterations, we return the subgraph with the highest shapley value from the leaf nodes for each $(u, v)$.

*(2) Random walks.* For each pair $(u, v)$ in $G$, we compute a subgraph $G_{\mathrm{sg}}(u, v)$ by the MCTS. Then we simulate a number of random walks from user $u$ on $G_{\mathrm{sg}}(u, v)$. Each random walk produces a path $\rho$; the path starts from $u$ and at each step, it either (a) terminates with $\alpha$ probability, or (b) moves to an out-neighbor of the current vertex with $(1 - \alpha)$ probability, where $\alpha$ is a predefined parameter. By composing the paths simulated from $G_{\mathrm{sg}}(u, v)$, we obtain a number of star patterns $Q_x$ to represent the properties of users. Similarly, by simulating the random walks from item $v$, we obtain a number of star patterns $Q_y$ for items. Taking each $Q_x$ and each $Q_y$ together, we get a dual pattern $Q[x_0, y_0] = \langle Q_x, Q_y \rangle$.

Putting MCTS and random walks together, RepsLearner identifies star patterns that retain the predictions of $\mathcal{M}$ models.

## Appendix D: Witness Computation

We focus on the following witness computation problem.

- *Input:* Graph $G$, a REP $\varphi$ and a user-item pair $(u, v)$.
- *Output:* The set of all witnesses of $\varphi$ at $(u, v)$.

Below we provide a PTIME algorithm for the witness computation problem, based on the notion of *bidirectional scattered matches*, from which we can restore the complete witnesses of $\varphi$. Specifically, for each $\rho$, the set $C_\rho$ of bidirectional scattered matches is computed by iteratively checking the refinement conditions of (inverse) scattered matches of all variables from $\rho$ in $G$ by *dynamic programming*.

As remarked earlier, when there is a variable predicate defined across two paths $\rho_x \in Q_x$ and $\rho_y \in Q_y$, the two paths are processed together, with an additional refinement condition. Otherwise, each path is processed independently. In the following, we first show how to compute the bidirectional scattered matches for a single path $\rho$ and then extend to the general case when two paths $\rho_x \in Q_x$

and $\rho_y \in Q_y$ are defined with a variable predicate at the leaves.

**Witness computation for a single path.** Assume *w.l.o.g.* that $\rho = (z_0, z_1, \ldots, z_n)$ is a given path from $Q_x$. Then we compute the set $C_\rho$ of bidirectional scattered matches as follows.

(1) We first initialize $C_{\rho_x}(z_0) = \{u\}$.

(2) We derive the inverse scattered matches top-down, from center to leaf along the path $\rho$ for all variables, by verifying the satisfaction of constant predicates (*i.e.,* refinement condition (b)) and checking edge connections and local 2-WL predicates on edges (*i.e.,* preserving the parent relationship as required in the refinement condition (c)). In particular, if $u$ does not induce any inverse scattered matches for the leaf $z_n$, it means that there is no witness of $\rho$ pivoted at $u$.

(3) Starting from the leaf $z_n$, we similarly process all variables bottom-up, until finding the scattered matches of the center $z_0$.

(4) We repeat Steps (2)-(3) until the bidirectional scattered matches become stable, *i.e.,* $C_\rho$ does not change anymore.

**Witness computation for a pair of paths.** Given two paths $\rho_x = (x_0, \ldots, x_{n_1}) \in Q_x$ and $\rho_y = (y_0, \ldots, y_{n_2}) \in Q_y$ pivoted at $(u, v)$, where a variable predicate $p_{\text{var}}$, *e.g.,* $x_{n_1}.A \oplus y_{n_2}.B$, is defined on the leaves of $\rho_x$ and $\rho_y$, we perform the following steps:

(1) Given $(u, v)$, we initialize $C_{\rho_x}(x_0) = \{u\}$ and $C_{\rho_y}(y_0) = \{v\}$.

(2) We derive the inverse scattered matches top-down, from center to leaf along the path $\rho_x$ (resp. $\rho_y$) for all variables, by verifying the satisfaction of constant predicates (*i.e.,* refinement condition (b)) and checking edge connections and local 2-WL predicates on edges (*i.e.,* preserving the parent relationship as required in the refinement condition (c)). In particular, if $u$ (resp. $v$) does not induce any inverse scattered matches for the leaf $x_{n_1}$ (resp. $y_{n_2}$), it means that there is no witness of $\rho_x$ (resp. $\rho_y$) pivoted at $u$ (resp. $(v)$)

(3) We conduct pairwise comparison between the inverse scattered matches of leaves $x_{n_1}$ and $y_{n_2}$, and remove the vertex pairs that do not satisfy $p_{\text{var}}$ in corresponding bidirectional scattered matches.

(4) Starting from the leaf $x_{n_1}$ (resp. $y_{n_2}$), we similarly process all variables bottom-up, from center to leaf along $\rho_x$ (resp. $\rho_y$) for all variables, until finding the scattered matches of the center $x_0$ (resp. $y_0$).

(5) We repeat Steps (2)-(4) until the bidirectional scattered matches become stable, *i.e.,* $C_{\rho_x}$ and $C_{\rho_y}$ do not change anymore.

**Theorem 5:** *For a graph $G$ and a* REP $\varphi$*, it is in* PTIME *to compute the set $C_\varphi$ of bidirectional scattered matches.* □

**Proof.** It suffices to show that the bidirectional scattered matches of each path $\rho$ (*i.e.,* $C_\rho$) of $Q$ in $\varphi$ can be computed in PTIME.

Consider the general case where two paths $\rho_x$ and $\rho_y$ are processed together, with a variable predicate defined on two leaves of $\rho_x$ and $\rho_y$; the case of a single path can be proven similarly.

Observe that the complexity is dominated by (a) computing inverse scattered matches top-down and (b) computing scattered matches bottom-up after validating the variable predicate on the matches of leaves. Although these steps are repeated until the bidirectional scattered matches are stable, it is shown that in the worst case, the number of repetitions is bounded by $O(|G||\rho|)$ since each variable has at most $|G|$ bidirectional scattered matches [14].

For part (a), since the pivots $(u, v)$ are given, we find the inverse scattered matches of other variables by fixing the match of $x_0$ and $y_0$ as $u$ and $v$, respectively, during top-down processing. Moreover, since the local 2-WL predicate on a pattern edge is compared with the given $(u, v)$, the verification of each local 2-WL predicate takes constant time (after pre-computing and indexing the link types from local 2-WL tests). This part takes $O(|G|(|\rho_x| + |\rho_y|))$ time.

For part (b), we pairwisely compare the inverse scattered matches obtained for two leaves to validate whether they satisfy the variable predicate, in $O(|G|^2)$ time. For vertex pairs $(w_x, w_y)$ passing the validation, we further apply the bottom-up procedure to verify that the given pivots $u$ and $v$ can induce $w_x$ and $w_y$, respectively. In particular, if $(u, v)$ do not induce such $(w_x, w_y)$, there is no witness of $\varphi$ at $(u, v)$. This part takes $O(|G|^2 + |G|(|\rho_x| + |\rho_y|))$ time.

Putting these together, we can see that it is in $O(c|G|^2 + c|G|(|\rho_x| + |\rho_y|))$ time to compute the set $C_\rho$ of bidirectional scattered matches, where $c = O(|G||\rho|)$ is the number of repetitions. This is in PTIME. Thus $C_\varphi$ can be computed in PTIME. □

## Appendix E: Anytime Explanations

Users may want to find successive explanations if they are not satisfied with the current ones; this is analogous to how we use search engines. In response to this we extend FETopk to an anytime algorithm, so that the users can retrieve the next top-$k$ explanations whenever needed, without recomputing from scratch. To enable this, we extend the evidences that we maintain in $\Psi$ and evaluate them in a *lazy* manner. Specifically, we make the following two changes:

(1) For each REP $\varphi$ in $\Sigma$, if $\hat{s}(\varphi)$ is less than the score of the $k$-th ranked evidence in $\Psi$ (*i.e.,* $T_k$), we do *not* directly stop the processing of $\varphi$ as we did in FETopk. Instead, we temporally keep $(\varphi, h_{\text{dummy}})$ as a *condensed* evidence in $\Psi$ with $\hat{s}(\varphi)$ as its key, where $h_{\text{dummy}}$ is a dummy match of $\varphi$; other evidences $(\varphi, h)$ in $\Psi$ are referred to as *complete* evidences in $\Psi$, when $h$ is a true witness via $\varphi$. Intuitively, although a condensed evidence cannot be a top-$k$ evidence at the current stage (since $\hat{s}(\varphi) < T_k$), it might make it when more evidences are needed. Thus, we keep them in $\Psi$ for later processing.

(2) When the users requires the next top-$k$ evidences in $\Psi$, we check whether the top-$k$ evidences in $\Psi$ are all complete evidences. If it is the case, we directly return them. Otherwise, we resume the processing of each condensed evidence by the descending order of the score upper bound $\hat{s}(\varphi)$ until all top-k evidences in $\Psi$ are complete evidences or there are no more condensed evidences to be processed. In this way, the condensed evidences are processed *lazily*, when they are likely to contribute to the top-$k$ evidences.

**Example 2:** Continuing with Example **??** where the re-computed $\hat{s}(\varphi_1)$ (*i.e.,* after removing $w_2$ and $w_4$) is less than $T_k$, instead of stopping the processing of $\varphi_1$, we keep $(\varphi_1, h_{\text{dummy}})$ as a condensed evidence in $\Psi$ and process the remaining REPs in $\Sigma$. Later, if a user requires the next top-$k$ in $\Psi$ and one of the top-$k$ in $\Psi$ is $(\varphi_1, h_{\text{dummy}})$, we resume the processing of $\varphi_1$, *e.g.,* by continuing to select $w_4$ with the maximum $\Delta_w$ as the next vertex to be processed. When all top-$k$ evidences in $\Psi$ are complete or there is no condensed evidence in $\Psi$, we return the top-$k$ evidences to the user. □